

Design and RTL Implementation of a RISC-V Based Spectrum Analyzer SoC with Hardware DSP Acceleration

INTRODUCTION

Problem Statement

Spectrum analysis is used to identify the frequency components present in a sampled signal. A typical spectrum-analysis process involves acquiring the signal, filtering unwanted components, and converting the signal from the time domain to the frequency domain using FFT. When these computationally intensive DSP operations are handled entirely by a processor, they can increase processing overhead and limit the efficiency of real-time analysis.

The proposed work addresses this by using a RISC-V based SoC with dedicated hardware DSP accelerators, where the processor handles system-level control while the computationally intensive signal-processing operations are performed using dedicated RTL hardware.

The proposed system aims to:

- Acquire digitized signal samples through an ADC interface.
- Filter unwanted noise and frequency components using an FIR filter.
- Analyze the frequency content using a dedicated FFT accelerator.
- Reduce processor workload by implementing DSP operations as hardware accelerators.
- Enable programmable control of the signal-processing system through the RISC-V processor.
- Provide processed spectrum information to an external host through UART.

Proposed SoC Architecture

The proposed system is designed as a **RISC-V based System-on-Chip (SoC)** for hardware-accelerated spectrum analysis. The **VeeR EL2 RISC-V processor** acts as the central controller and communicates with the memories, peripherals, and application-specific IPs through the bus interconnect. The architecture follows a hardware-software co-design approach, where system configuration, control, and result handling are performed by the processor, while computationally intensive DSP operations are offloaded to dedicated RTL hardware accelerators.

The signal-processing path consists of **ADC Interface** → **FIR Filter** → **FFT Accelerator**. The ADC interface receives digitized samples from an external ADC and

feeds them to the FIR filter for signal conditioning. The filtered samples are then provided to the FFT accelerator, which transforms the signal from the time domain into its frequency-domain representation. The resulting spectral information can then be accessed by the RISC-V processor and transmitted to an external host through UART.

The SoC consists of the following components:

- **VeeR EL2 RISC-V Processor**
 - Serves as the central controller of the SoC.
 - Configures and controls the application-specific IPs.
 - Initiates signal acquisition and processing.
 - Accesses and interprets the generated spectral data.
- **Instruction Memory**
 - Stores the program executed by the RISC-V processor.
 - Contains the software responsible for configuring and controlling the spectrum-analysis system.
- **Data Memory**
 - Stores processed DSP data, including FFT results.
 - Provides storage for FFT output/spectrum data and software-accessible processing data.
 - Makes generated spectrum results accessible to the RISC-V processor.
- **Bus Interconnect**
 - Provides communication between the RISC-V processor, memories, peripherals, and custom DSP IPs.
 - Enables the application-specific accelerators to be accessed as part of the SoC.
- **UART:** Transmits processed spectrum information, such as dominant frequency components and system status, to an external host.
- **Timer:** Provides timing and periodic control for signal acquisition and processing operations.
- **GPIO:** Provides simple external control and status indication, such as acquisition or processing completion.

Selected IPs

1. ADC Interface

- Receives digitized samples from the external ADC and feeds them into the SoC.
- Spectrum analysis requires a reliable interface for acquiring the sampled input signal before digital processing.

2. FIR Filter

- Performs digital filtering of the acquired signal to attenuate unwanted frequency components and noise.
- Filtering the input signal before FFT improves the quality of the spectrum and provides a dedicated hardware implementation of a fundamental DSP operation.

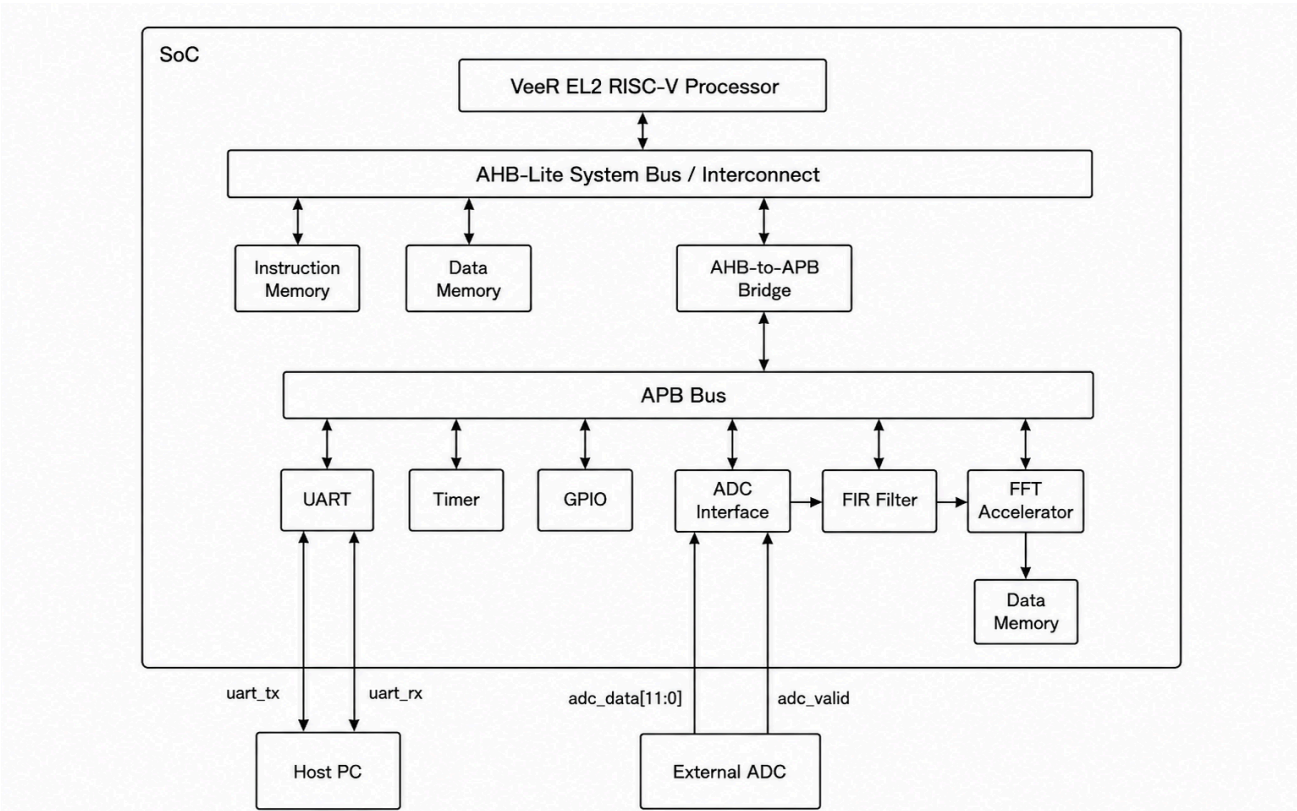
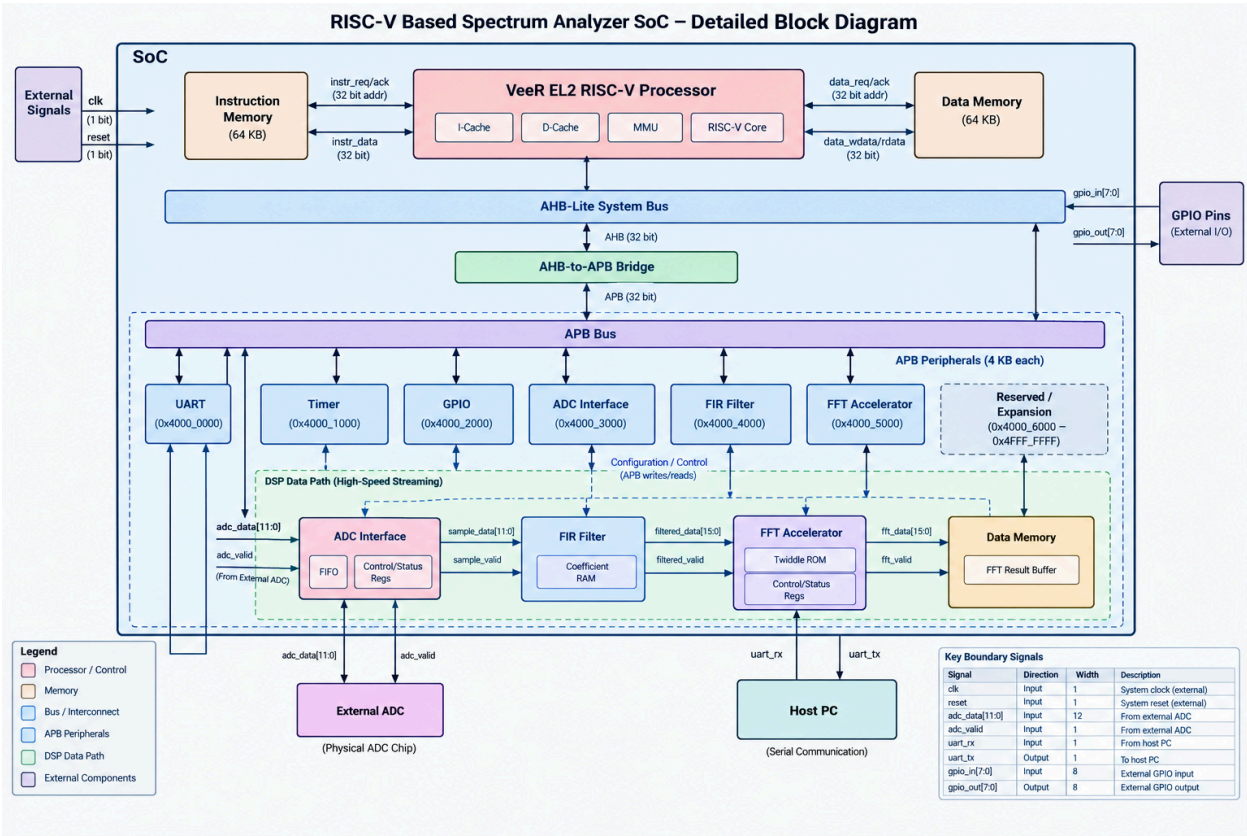
3. FFT Accelerator

- Converts the filtered time-domain samples into their frequency-domain representation.
- FFT is the core computation required for spectrum analysis and is computationally intensive, making it well suited for dedicated hardware acceleration.

Process Flow

External analog Signal → External ADC → ADC Interface → FIR Filter → FFT
Accelerator → Data Memory ↔ Bus Interconnect ↔ RISC-V (VeeR EL2) ↔ Instruction
Memory, Timer, GPIO, UART → Host PC

BLOCK DIAGRAM



SIGNAL LIST

1. VeeR EL2 RISC-V Processor — Signal List

VeeR EL2 provides separate AHB-Lite interfaces for **instruction fetch** and **load/store/data access**. The PRM specifies these as 32-bit address and 64-bit data interfaces.

A. Instruction Fetch AHB-Lite Interface

Signal	Direction	Source → Destination	Width (bits)	Description
<code>haddr[31:0]</code>	Output	VeeR → AHB Bus	32	Instruction memory address
<code>hburst[2:0]</code>	Output	VeeR → AHB Bus	3	Burst type; hardwired to <code>000</code>
<code>hmastlock</code>	Output	VeeR → AHB Bus	1	Locked-transfer indication; hardwired to <code>0</code>
<code>hprot[3:0]</code>	Output	VeeR → AHB Bus	4	Transfer protection attributes
<code>hsize[2:0]</code>	Output	VeeR → AHB Bus	3	Instruction transfer size
<code>htrans[1:0]</code>	Output	VeeR → AHB Bus	2	AHB transfer type
<code>hwrite</code>	Output	VeeR → AHB Bus	1	Indicates read/write transfer; instruction fetch is read
<code>hrdata[63:0]</code>	Input	AHB Bus → VeeR	64	Instruction data returned from memory
<code>hready</code>	Input	AHB Bus → VeeR	1	Indicates completion of the transfer
<code>hresp</code>	Input	AHB Bus → VeeR	1	Indicates AHB transfer response/error

The VeeR PRM explicitly lists these instruction-fetch AHB-Lite signals and their widths.

B. Load/Store Unit (Data) AHB-Lite Interface

Signal	Direction	Source → Destination	Width (bits)	Description
lsu_haddr[31:0]	Output	VeeR → AHB Bus	32	Address for data/peripheral access
lsu_hburst[2:0]	Output	VeeR → AHB Bus	3	Burst type; hardwired to 000
lsu_hmastlock	Output	VeeR → AHB Bus	1	Locked-transfer indication; hardwired to 0
lsu_hprot[3:0]	Output	VeeR → AHB Bus	4	Transfer protection attributes
lsu_hsize[2:0]	Output	VeeR → AHB Bus	3	Data transfer size
lsu_htrans[1:0]	Output	VeeR → AHB Bus	2	AHB transfer type
lsu_hwdata[63:0]	Output	VeeR → AHB Bus	64	Data written to memory/peripherals
lsu_hwrite	Output	VeeR → AHB Bus	1	Indicates read or write transfer
lsu_hrdata[63:0]	Input	AHB Bus → VeeR	64	Data returned from memory/peripheral
lsu_hready	Input	AHB Bus → VeeR	1	Indicates completion of the transfer
lsu_hresp	Input	AHB Bus → VeeR	1	Indicates AHB transfer response/error

These are directly specified in the VeeR EL2 PRM.

C. Clock, Reset and Interrupt — Relevant Core Signals

Signal	Direction	Source → Destination	Width	Description
<code>clk</code>	Input	SoC clock → VeeR	1	System clock
<code>rst_l</code>	Input	SoC reset → VeeR	1	Active-low core-complex reset
External interrupt input	Input	PIC/peripherals → VeeR	1*	Signals an external interrupt to the processor

*The exact interrupt implementation/port naming depends on how the VeeR PIC is instantiated. VeeR supports a configurable PIC with **2–255 external interrupt sources**, so we should finalize this signal when we design our interrupt architecture.

The VeeR documentation specifies `rst_l` as the core-complex reset and notes that the clock must be stable before reset is deasserted.

2. Instruction Memory — Signal List

For our project, Instruction Memory is a **64 KB read-only program memory** mapped at `0x0000_0000 – 0x0000_FFFF`. Its job is simply to store the firmware/program executed by the VeeR EL2 processor. The VeeR instruction-fetch interface is an AHB-Lite master interface, with a **32-bit address bus** and **64-bit read-data bus**.

One important point: **Instruction Memory does not need a separate write interface in our SoC architecture**. The processor only fetches instructions from it during normal operation.

AHB-Lite interface

Signal	Direction	Source → Destination	Width (bits)	Description
<code>haddr[31:0]</code>	Input	VeeR → Instruction Memory	32	Address of instruction being fetched
<code>htrans[1:0]</code>	Input	VeeR → Instruction Memory	2	Indicates type/state of AHB transfer
<code>hsize[2:0]</code>	Input	VeeR → Instruction Memory	3	Indicates transfer size
<code>hwrite</code>	Input	VeeR → Instruction Memory	1	Indicates read/write; instruction memory uses read access
<code>hburst[2:0]</code>	Input	VeeR → Instruction Memory	3	Burst type; VeeR hardwires this to <code>000</code>
<code>hmastlock</code>	Input	VeeR → Instruction Memory	1	Locked-transfer indication; hardwired to <code>0</code>
<code>hprot[3:0]</code>	Input	VeeR → Instruction Memory	4	Transfer protection attributes
<code>hrdata[63:0]</code>	Output	Instruction Memory → VeeR	64	Instruction data returned to processor

hready	Output	Instruction Memory → VeeR	1	Indicates that instruction read has completed
hresp	Output	Instruction Memory → VeeR	1	Indicates successful transfer or error

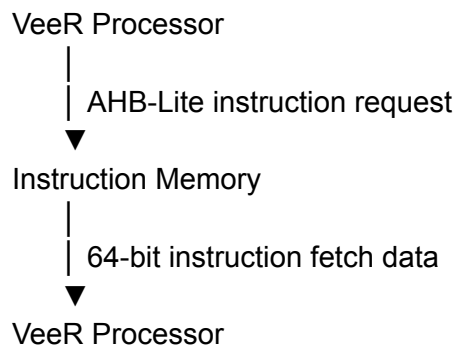
These signal names and widths follow the VeeR EL2 instruction-fetch AHB-Lite interface.

Clock and reset

Signal	Direction	Source → Destination	Width (bits)	Description
clk	Input	SoC → Instruction Memory	1	System clock
reset	Input	SoC → Instruction Memory	1	Initializes memory interface/control logic

How it works in our SoC

The basic operation is:



So, for example:

1. VeeR generates an instruction address through **haddr**.
2. Instruction Memory decodes that address within its **0x0000_0000–0x0000_FFFF** range.
3. The corresponding memory contents are returned through **hrdata[63:0]**.
4. **hready** indicates completion.
5. **hresp** indicates whether the access succeeded.

One thing to be very clear about

64-bit `hrdata` does NOT mean our RISC-V instructions are 64 bits wide. It means the VeeR EL2 instruction-fetch interface transfers data over a 64-bit AHB-Lite data channel. The processor can extract the required instruction(s) from the fetched data.

Also, our **64 KB size is our SoC design choice**, not a mandatory VeeR requirement. The VeeR documentation allows configurable instruction-memory sizes including 64 KB.

VeeR IFU → AHB-Lite request signals → Instruction Memory → `hrdata[63:0]` + `hready` + `hresp` → VeeR IFU

3. Data Memory — Signal List

Our current architecture allocates:

- **Base:** 0x1000_0000
- **Size:** 64 KB
- **End:** 0x1000_FFFF

A. AHB-Lite Interface

Signal	Direction	Source → Destination	Width (bits)	Description
lsu_haddr[31:0]	Input	VeeR/Bus → Data Memory	32	Address of the data-memory access
lsu_htrans[1:0]	Input	VeeR/Bus → Data Memory	2	Indicates the AHB transfer type
lsu_hsize[2:0]	Input	VeeR/Bus → Data Memory	3	Indicates size of the data transfer
lsu_hwrite	Input	VeeR/Bus → Data Memory	1	Selects read (0) or write (1) operation
lsu_hburst[2:0]	Input	VeeR/Bus → Data Memory	3	Specifies AHB burst type
lsu_hmastlock	Input	VeeR/Bus → Data Memory	1	Indicates locked transfer

<code>lsu_hprot[3:0]</code>	Input	VeeR/Bus → Data Memory	4	Specifies access protection attributes
<code>lsu_hwdata[63:0]</code>	Input	VeeR/Bus → Data Memory	64	Data supplied for a write operation
<code>lsu_hrddata[63:0]</code>	Output	Data Memory → VeeR/Bus	64	Data returned for a read operation
<code>lsu_hready</code>	Output	Data Memory → VeeR/Bus	1	Indicates completion of the memory transfer
<code>lsu_hresp</code>	Output	Data Memory → VeeR/Bus	1	Indicates successful transfer or error

These correspond to the VeeR EL2 load/store AHB-Lite interface, whose address channel is 32-bit and data channel is 64-bit.

B. Clock and Reset

Signal	Direction	Source → Destination	Width (bits)	Description
<code>clk</code>	Input	SoC → Data Memory	1	System clock for memory operation
<code>reset</code>	Input	SoC → Data Memory	1	Initializes memory interface/control logic

C. What happens during a read?

Suppose the RISC-V firmware wants to read some stored FFT result.

RISC-V

|

| Address + read request



AHB-Lite

|



Data Memory

|

| Read data



AHB-Lite

|



RISC-V

The important signals are:

lsu_haddr

lsu_htrans

lsu_hsize

lsu_hwrite = 0



Data Memory



Isu_hrddata

Isu_hready

Isu_hresp

D. What happens during a write?

For example, the processor wants to store a processed result:

RISC-V



| Address + write data



AHB-Lite



Data Memory

The important signals are:

Isu_haddr

Isu_htrans

Isu_hsize

Isu_hwrite = 1

Isu_hwdata



Data Memory



lsu_hready

lsu_hresp

There's something we need to **keep separate** in our specification.

We previously described:

ADC → FIR → FFT → Data Memory

as the **DSP data path**.

But the signals above describe:

VeeR LSU ↔ AHB-Lite ↔ Data Memory

These are **two different access paths**.

The DSP blocks will eventually need their own **data-path interface signals** to send/store samples and FFT results. We should **NOT add those signals to the Data Memory table yet**, because we haven't finalized how our DSP-to-memory connection will actually be implemented.

That is something we'll design when we reach the **cross-block/data-path section**.

So for now:

Processor/Memory path

VeeR LSU



AHB-Lite



Data Memory

DSP path

ADC Interface



FIR



FFT



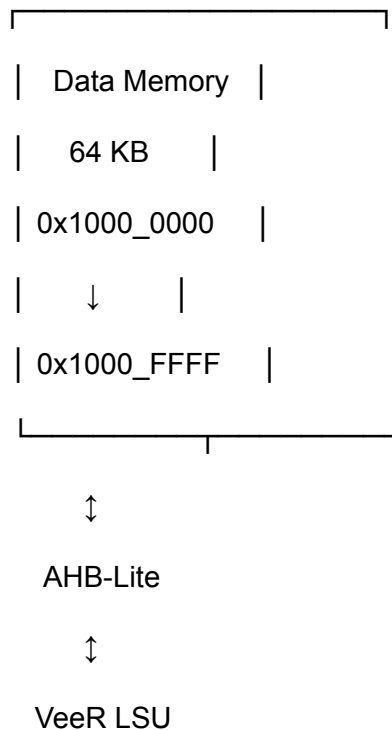
Data storage

We'll later decide exactly **how the second path reaches Data Memory**.

This is important because we don't want to accidentally claim that the DSP blocks can directly write the AHB Data Memory when our RTL hasn't yet defined that interface.

Final Block-3 picture

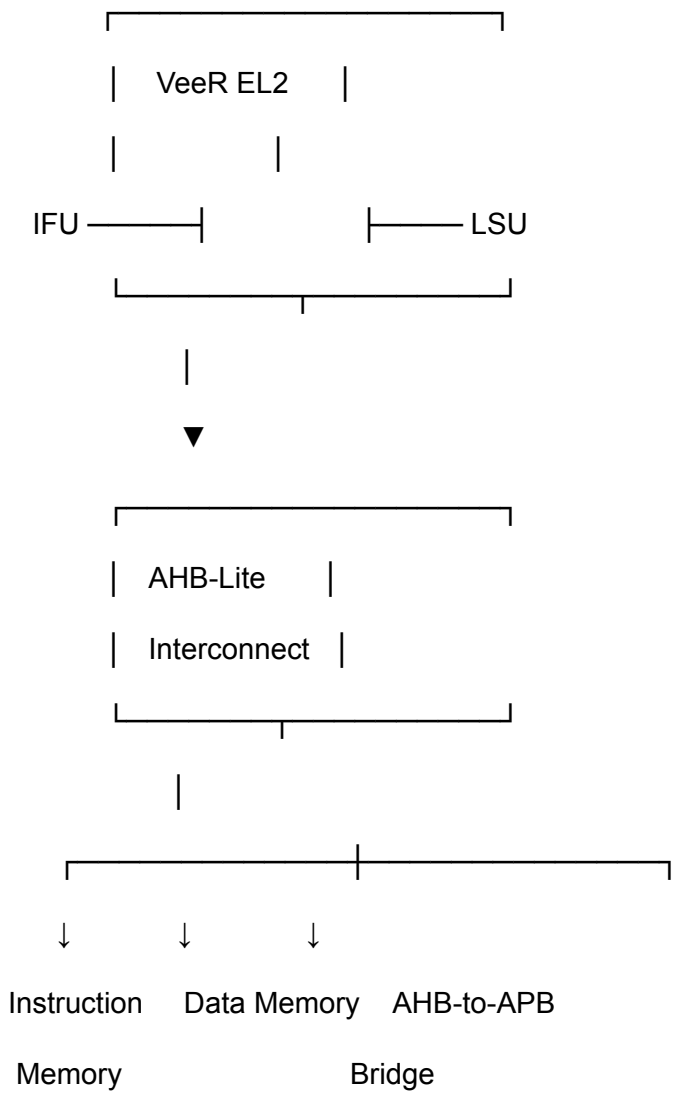
For your signal-list document, the current Data Memory interface is:



4. AHB-Lite System Bus / Interconnect — Signal List

For our SoC, the interconnect receives the VeeR's **instruction-fetch and load/store AHB transactions**, decodes the address, and routes each transaction to the appropriate target.

Conceptually:



A. Processor → AHB Interconnect

Because VeeR has separate IFU and LSU interfaces, there are two transaction groups.

Signal	Direction	Source → Destination	Width	Description
--------	-----------	----------------------	-------	-------------

<code>haddr[31:0]</code>	Input	VeeR IFU → Interconnect	32	Instruction access address
<code>htrans[1:0]</code>	Input	VeeR IFU → Interconnect	2	Instruction AHB transfer type
<code>hsize[2:0]</code>	Input	VeeR IFU → Interconnect	3	Instruction transfer size
<code>hwrite</code>	Input	VeeR IFU → Interconnect	1	Indicates read/write operation
<code>hburst[2:0]</code>	Input	VeeR IFU → Interconnect	3	Burst type
<code>hmastlock</code>	Input	VeeR IFU → Interconnect	1	Locked-transfer indication
<code>hprot[3:0]</code>	Input	VeeR IFU → Interconnect	4	Transfer protection attributes

For the LSU:

Signal	Direction	Source → Destination	Width	Description
<code>lsu_haddr[31:0]</code>	Input	VeeR LSU → Interconnect	32	Data/peripheral access address

<code>lsu_htrans[1:0]</code>	Input	VeeR LSU → Interconnect	2	Data AHB transfer type
<code>lsu_hsize[2:0]</code>	Input	VeeR LSU → Interconnect	3	Data transfer size
<code>lsu_hwrite</code>	Input	VeeR LSU → Interconnect	1	Indicates read/write operation
<code>lsu_hburst[2:0]</code>	Input	VeeR LSU → Interconnect	3	Burst type
<code>lsu_hmastlock</code>	Input	VeeR LSU → Interconnect	1	Locked-transfer indication
<code>lsu_hprot[3:0]</code>	Input	VeeR LSU → Interconnect	4	Transfer protection attributes
<code>lsu_hwdata[63:0]</code>	Input	VeeR LSU → Interconnect	64	Data supplied during a write

These widths follow the VeeR EL2 AHB-Lite interfaces documented for the IFU and LSU.

B. Interconnect → Processor Response Signals

The interconnect routes the selected target's response back to VeeR.

Instruction side

Signal	Direction	Source → Destination	Width	Description
hrdata[63:0]	Output	Interconnect → VeeR IFU	64	Instruction data returned from memory
hready	Output	Interconnect → VeeR IFU	1	Indicates instruction transfer completion
hresp	Output	Interconnect → VeeR IFU	1	Indicates instruction transfer response

LSU side

Signal	Direction	Source → Destination	Width	Description
lsu_hrdata[63:0]	Output	Interconnect → VeeR LSU	64	Data returned from memory/peripheral
lsu_hready	Output	Interconnect → VeeR LSU	1	Indicates data transfer completion
lsu_hresp	Output	Interconnect → VeeR LSU	1	Indicates data transfer response

C. Address Decode / Target Select Signals

This is where the interconnect becomes important to **our particular SoC**.

The interconnect looks at the address and determines where the transaction should go.

Our current map is:

0x0000_0000 – 0x0000_FFFF → Instruction Memory

0x1000_0000 – 0x1000_FFFF → Data Memory

0x4000_0000 – 0x4FFF_FFFF → APB peripheral region

Therefore, internally we will have target-selection signals such as:

Signal	Direction	Source → Destination	Width	Description
<code>imem_sel</code>	Output	Interconnect → Instruction Memory	1	Selects instruction-memory region
<code>dmem_sel</code>	Output	Interconnect → Data Memory	1	Selects data-memory region
<code>apb_sel</code>	Output	Interconnect → AHB-to-APB Bridge	1	Selects APB peripheral region

These names are our architectural names. They are not VeeR-required signal names; the exact RTL names will depend on how we implement the interconnect.

D. Address Routing Signals

The selected target needs the relevant AHB transaction information.

So the interconnect effectively distributes:

Address

Transfer type

Transfer size

Read/write

Write data

Protection/control

to the selected target.

For example:

VeeR LSU

|

| lsu_haddr = 0x1000_0040



AHB Interconnect

|

| Address decoder

|

| → dmem_sel = 1

|



Data Memory

Or:

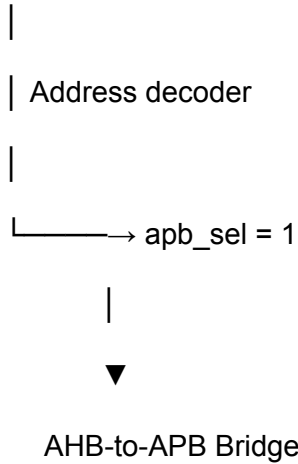
VeeR LSU

|

| lsu_haddr = 0x4000_3000



AHB Interconnect



E. Clock and Reset

Signal	Direction	Source → Destination	Width	Description
<code>clk</code>	Input	SoC → AHB Interconnect	1	System clock
<code>reset</code>	Input	SoC → AHB Interconnect	1	Resets interconnect control/decoding logic

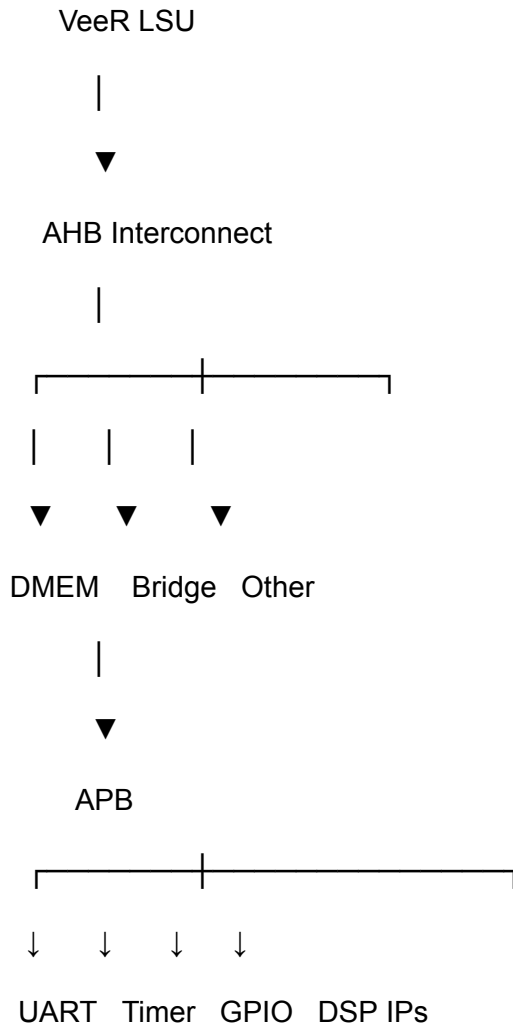
The most important thing to understand

The AHB-Lite interconnect is **not performing DSP processing**.

Its job is basically:

"The processor generated an address. Which block owns this address, and where should I send the transaction?"

For example:



So when RISC-V executes something like a memory-mapped write to:

0x4000_5000

the interconnect recognizes:

0x4000_5000 → FFT Accelerator

and sends that transaction toward the **AHB-to-APB bridge**, which eventually produces the corresponding APB transaction for the FFT registers.

That is how your RISC-V can control the FFT **without being directly wired to the FFT's internal control logic**.

 One architectural detail we'll resolve later

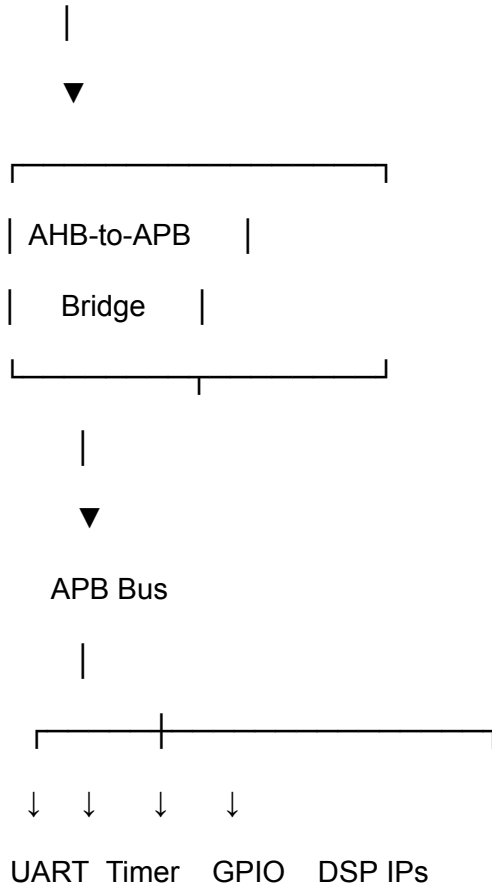
Because your VeeR has **two AHB-Lite interfaces (IFU and LSU)**, our final RTL interconnect needs to handle both. The IFU primarily targets Instruction Memory, while the LSU accesses Data Memory and memory-mapped peripherals.

We therefore shouldn't blindly copy a generic "single AHB master → multiple slaves" diagram into the final implementation.

For the **specification document**, the above is the correct level. When we get to implementation, we'll decide exactly how the two VeeR interfaces are connected/decoded.

5. AHB-to-APB Bridge — Signal List

VeeR / AHB-Lite



The key idea is:

AHB signals come into the bridge → the bridge converts the transaction into APB signals → the selected peripheral responds → the bridge converts the response back to AHB.

A. AHB-Lite Side — Inputs to Bridge

Signal	Direction	Source → Destination	Width (bits)	Description
<code>haddr[31:0]</code>	Input	AHB Interconnect → Bridge	32	Address of the peripheral access
<code>htrans[1:0]</code>	Input	AHB Interconnect → Bridge	2	Indicates the AHB transfer type
<code>hwrite</code>	Input	AHB Interconnect → Bridge	1	Selects read or write operation
<code>hsize[2:0]</code>	Input	AHB Interconnect → Bridge	3	Indicates transfer size
<code>hburst[2:0]</code>	Input	AHB Interconnect → Bridge	3	Indicates AHB burst type
<code>hmastlock</code>	Input	AHB Interconnect → Bridge	1	Indicates locked transfer
<code>hprot[3:0]</code>	Input	AHB Interconnect → Bridge	4	Transfer protection attributes
<code>hwddata[63:0]</code>	Input	AHB Interconnect → Bridge	64	Data supplied during a write

For our VeeR-based system, these are derived from the processor's LSU AHB-Lite transaction.

B. AHB-Lite Side — Outputs from Bridge

Signal	Direction	Source → Destination	Width (bits)	Description
<code>hrdata[63:0]</code>	Output	Bridge → AHB Interconnect	64	Data returned from the APB peripheral
<code>hready</code>	Output	Bridge → AHB Interconnect	1	Indicates completion of the AHB transaction
<code>hresp</code>	Output	Bridge → AHB Interconnect	1	Indicates successful or erroneous AHB transfer

So the bridge essentially converts:

AHB Request

↓

Bridge

↓

APB transaction

↓

Peripheral response

↓

Bridge

↓

AHB Response

C. APB Side — Outputs from Bridge

These are the signals generated by the bridge toward the APB bus.

Signal	Direction	Source → Destination	Width (bits)	Description
<code>paddr[31:0]</code>	Output	Bridge → APB Bus	32	Address of the selected peripheral register
<code>pwrdata[31:0]</code>	Output	Bridge → APB Bus	32	Write data sent to peripheral
<code>pwrite</code>	Output	Bridge → APB Bus	1	Indicates APB write (<code>1</code>) or read (<code>0</code>)
<code>psel / psel_x</code>	Output	Bridge → APB Bus	1 / N	Selects the target APB peripheral
<code>penable</code>	Output	Bridge → APB Bus	1	Indicates APB ACCESS phase

About `psel`

This is important.

Because you have **six APB peripherals**, we will ultimately have either:

`psel_uart`

`psel_timer`

psel_gpio

psel_adc

psel_fir

psel_fft

or a decoded **PSEL[N-1:0]** vector.

For the specification, I recommend documenting it as:

psel[N-1:0] — one-hot peripheral-select signals generated by the APB decoder.

With six peripherals:

PSEL[5:0]

where each bit selects one peripheral.

D. APB Side — Inputs to Bridge

The selected APB peripheral sends these back.

Signal	Direction	Source → Destination	Width (bits)	Description
prdata[31:0]	Input	APB Peripheral → Bridge	32	Read data returned by selected peripheral
pready	Input	APB Peripheral → Bridge	1	Indicates completion of APB transfer

<code>pslverr</code>	Input	APB Peripheral → Bridge	1	Indicates an APB transfer error
----------------------	-------	-------------------------	---	---------------------------------

E. Clock and Reset

Signal	Direction	Source → Destination	Width (bits)	Description
<code>pclk / clk</code>	Input	SoC → Bridge	1	Clock used by the bridge and APB subsystem
<code>presetn / reset</code>	Input	SoC → Bridge	1	Resets bridge state/control logic

We will eventually **freeze the exact reset naming** when we decide how the APB subsystem is instantiated.

The actual conversion

Suppose RISC-V wants to write to:

`0x4000_5000`

which is our FFT Accelerator region.

The flow becomes:

RISC-V

|

| AHB write

| Address = 0x4000_5000



AHB Interconnect



AHB-to-APB Bridge



|— PADDR = 0x4000_5000

|— PWDATA = data

|— PWRITE = 1

|— PSEL = FFT selected

└— PENABLE



FFT Accelerator

The FFT then responds:

FFT



|— PRDATA

|— PREADY

└— PSLVERR



AHB-to-APB Bridge

|



AHB Interconnect

|



RISC-V

★ APB transaction phases

The bridge must follow the APB transaction sequence:

IDLE



SETUP



ACCESS



IDLE

During **SETUP**:

PSEL = 1

PENABLE = 0

During **ACCESS**:

PSEL = 1

PENABLE = 1

The bridge waits for:

PREADY = 1

and then completes the AHB transaction.

One important correction for our specification

Our **AHB side is 64-bit**, because that's what VeeR's LSU interface provides.

Our APB peripheral interface is currently planned as **32-bit**.

Therefore the bridge is also responsible for handling the **AHB 64-bit ↔ APB 32-bit data-width relationship**.

This becomes important when we later define:

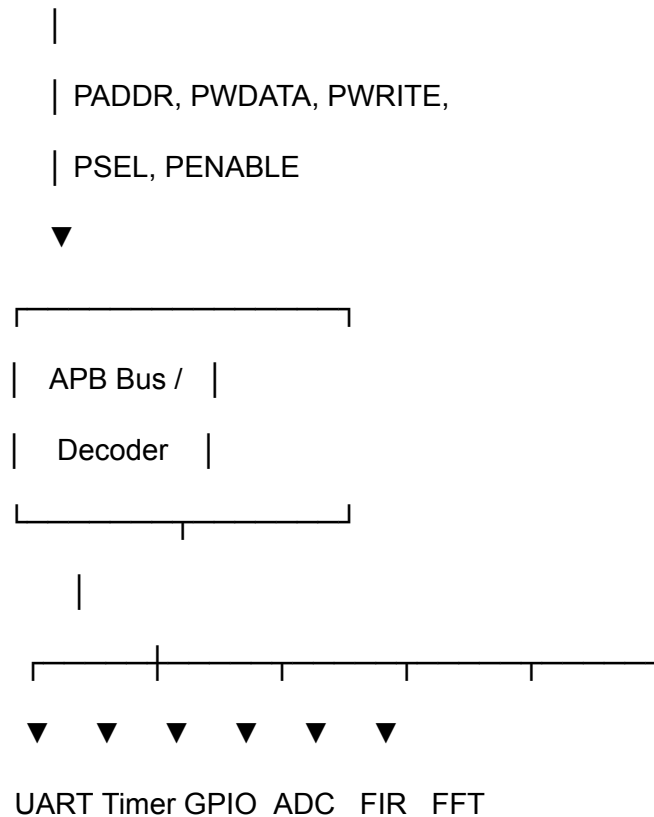
- register widths
- **HSIZE**
- byte enables
- which half of **HWDATA[63:0]** is used
- how **PRDATA[31:0]** is returned in **HRDATA[63:0]**

We should not invent those details yet. We'll explicitly define the data-width conversion rules in the bus-protocol section once we freeze the register architecture.

6. APB Bus / Decoder — Signal List

This is the block that takes the APB transaction generated by the AHB-to-APB bridge and **selects exactly one of our six APB peripherals based on the address.**

AHB-to-APB Bridge



Our six peripheral address regions are:

0x4000_0000 → UART

0x4000_1000 → Timer

0x4000_2000 → GPIO

0x4000_3000 → ADC Interface

0x4000_4000 → FIR Filter

0x4000_5000 → FFT Accelerator

A. Common APB Bus Signals

These come from the AHB-to-APB bridge.

Signal	Direction	Source → Destination	Width (bits)	Description
<code>paddr[31:0]</code>	Input	Bridge → APB Decoder	32	Address of the APB register being accessed
<code>pdata[31:0]</code>	Input	Bridge → APB Decoder	32	Data supplied during an APB write
<code>pwrite</code>	Input	Bridge → APB Decoder	1	Indicates APB write (1) or read (0)
<code>penable</code>	Input	Bridge → APB Peripherals	1	Indicates APB ACCESS phase
<code>pse1</code>	Input	Bridge → APB Decoder	1	Indicates an active APB transaction
<code>pclk</code>	Input	SoC → APB Bus	1	APB clock
<code>presetn</code>	Input	SoC → APB Bus	1	Active-low APB reset

B. Peripheral Select Signals

The decoder examines **paddr** and generates individual peripheral-select signals.

Signal	Direction	Source → Destination	Width	Description
psel_uart	Output	APB Decoder → UART	1	Selects UART when address falls in UART region
psel_timer	Output	APB Decoder → Timer	1	Selects Timer
psel_gpio	Output	APB Decoder → GPIO	1	Selects GPIO
psel_adc	Output	APB Decoder → ADC Interface	1	Selects ADC Interface
psel_fir	Output	APB Decoder → FIR	1	Selects FIR Filter
psel_ffft	Output	APB Decoder → FFT	1	Selects FFT Accelerator

Only the peripheral corresponding to the requested address should be selected for a valid transaction.

For example:

PADDR = 0x4000_4000



APB Address Decoder

↓

psel_fir = 1

all others = 0

C. APB Data/Response Return Path

The selected peripheral returns the result of the transaction.

Signal	Direction	Source → Destination	Width (bits)	Description
<code>prdata[31:0]</code>	Output	Selected Peripheral → APB Decoder/Bridge	32	Read data returned by selected peripheral
<code>pready</code>	Output	Selected Peripheral → APB Decoder/Bridge	1	Indicates completion of APB transaction
<code>pslverr</code>	Output	Selected Peripheral → APB Decoder/Bridge	1	Indicates an APB transfer error

The decoder/mux routes the response from the **selected peripheral** back to the bridge.

Conceptually:

UART —┐

Timer —┐

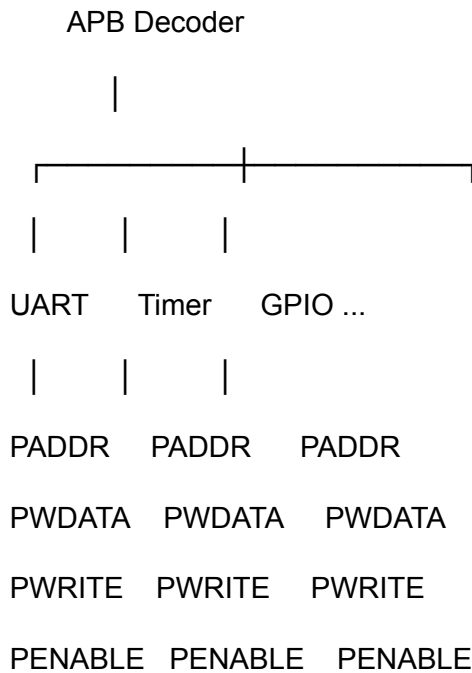
GPIO —┐

ADC —┐

FIR —┐ —→ Response MUX → Bridge

D. APB Peripheral Data/Control Distribution

The common APB signals are distributed to the selected peripheral:



Therefore, the following signals are effectively **common APB signals available to each peripheral**:

- PADDR[31:0]
- PWDATA[31:0]
- PWRITE
- PENABLE
- PCLK
- PRESETn

while each peripheral gets its own **PSEL**.

E. Address Decode Logic

The decoder uses the upper address bits to determine the target.

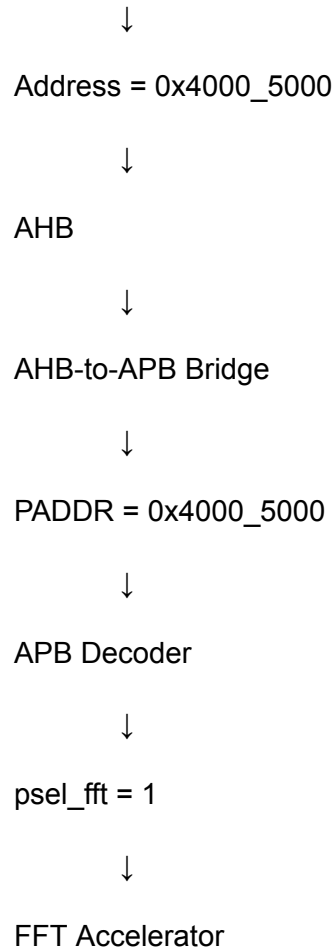
For our current map:

Address range	Select
0x4000_0000 – 0x4000_0FFF	pse1_uar t
0x4000_1000 – 0x4000_1FFF	pse1_tim er
0x4000_2000 – 0x4000_2FFF	pse1_gpi o
0x4000_3000 – 0x4000_3FFF	pse1_adc
0x4000_4000 – 0x4000_4FFF	pse1_fir
0x4000_5000 – 0x4000_5FFF	pse1_fft

So the decoder doesn't need complicated logic.

For example:

RISC-V wants to write FFT control register



One important distinction

Don't confuse:

PSEL

with:

PSEL_UART, PSEL_TIMER, etc.

The bridge generates the **overall APB transaction**, while the decoder generates the **individual peripheral selection**.

Conceptually:

Bridge

|
| PSEL

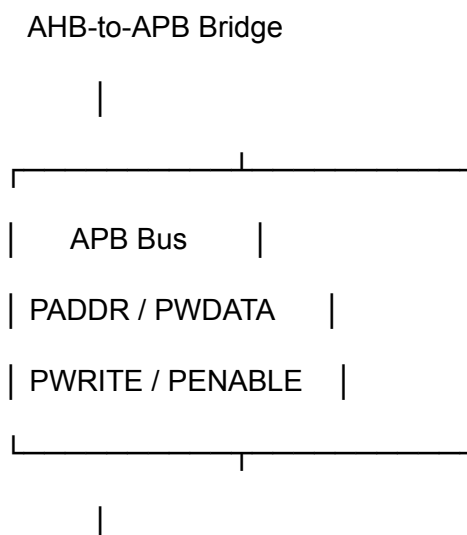


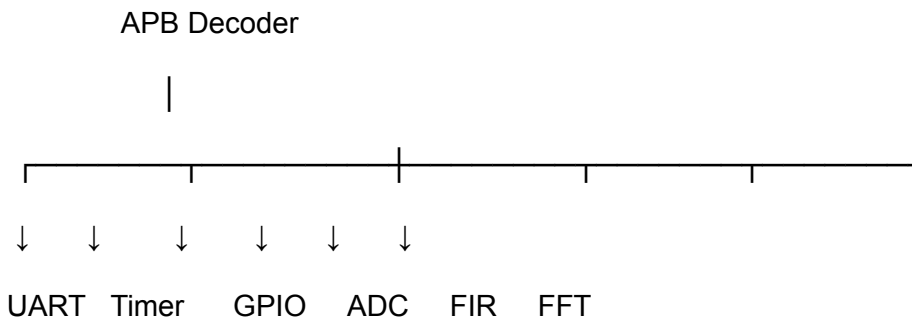
APB Decoder

|
|— PSEL_UART
|— PSEL_TIMER
|— PSEL_GPIO
|— PSEL_ADC
|— PSEL_FIR
|— PSEL_FFT

Depending on our eventual RTL implementation, we may combine the decoder with the APB interconnect rather than instantiate it as a physically separate module. **For the specification document, however, treating it as the APB Bus/Decoder is perfectly clear.**

Final Block-6 picture

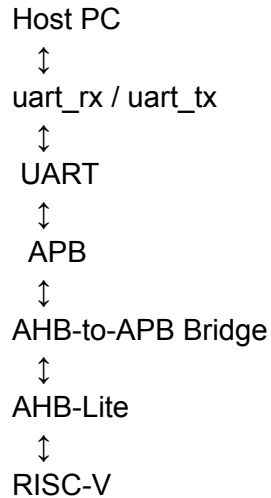




So **Block 6** is essentially responsible for address decoding, peripheral selection, APB signal distribution, and response multiplexing.

7. UART — Signal List

UART is one of the **standard/internal IPs** in our SoC. In our project, its main practical role is to provide a simple communication path between the **RISC-V SoC and the Host PC**.



We need to document **three categories of UART signals**:

1. APB interface signals
2. External UART signals
3. Interrupt/status-related signals

A. APB Interface

These are the signals through which RISC-V configures and accesses UART registers.

Signal	Direction	Source → Destination	Width (bits)	Description
PCLK	Input	SoC → UART	1	APB clock
PRESETn	Input	SoC → UART	1	Active-low UART reset
PADDR[31:0]	Input	APB Bus → UART	32	Address of UART register
PSEL	Input	APB Decoder → UART	1	Selects UART for APB transaction

PENABLE	Input	APB Bus → UART	1	Indicates APB access phase
PWRITE	Input	APB Bus → UART	1	Indicates register read/write
PWDATA[31:0]	Input	APB Bus → UART	32	Data written to UART register
PRDATA[31:0]	Output	UART → APB Bus	32	Data read from UART register
PREADY	Output	UART → APB Bus	1	Indicates completion of APB transaction
PSLVERR	Output	UART → APB Bus	1	Indicates APB access error

B. External UART Signals

These are the signals that actually cross the **SoC boundary** toward the Host PC.

Signal	Direction	Source → Destination	Width (bits)	Description
uart_rx	Input	Host PC → UART	1	Serial data received by SoC
uart_tx	Output	UART → Host PC	1	Serial data transmitted from SoC

These are **1-bit signals** because UART communication is serial: one bit is transmitted/received at a time.

C. UART Interrupt

The UART can notify the processor when an event requiring CPU attention occurs.

Signal	Direction	Source → Destination	Width (bits)	Description
<code>uart_irq</code>	Output	UART → Interrupt Controller/PIC	1	Indicates a UART interrupt condition

For our project, we can use this for events such as:

- Receive data available
- Transmit operation completed
- UART error condition

The exact interrupt sources we finally implement will depend on our UART RTL. We shouldn't claim all three unless our selected UART IP actually supports them.

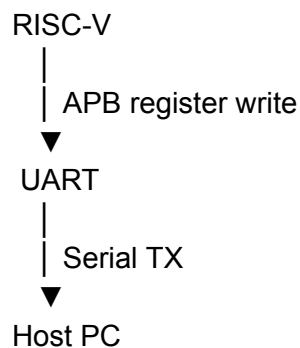
D. Clock/Reset

We already included `PCLK` and `PRESETn` above because they are part of the APB interface, so we don't need to duplicate them in another table.

How UART actually works in OUR project

RISC-V → Host PC

Suppose the FFT is complete and the firmware wants to send a result/status message.

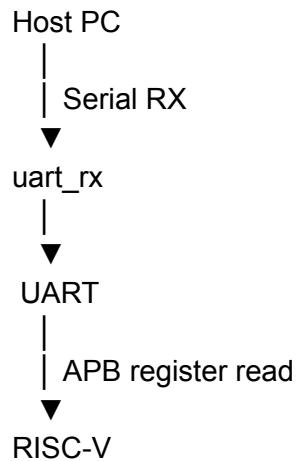


The processor writes data into a UART transmit register. The UART converts that parallel register data into a serial stream on:

uart_tx

Host PC → RISC-V

For incoming data:



The UART receives the serial stream, reconstructs the data and makes it available through its registers.

★ What UART is NOT doing

This is important for explaining your project to faculty.

UART is **not part of the spectrum-analysis processing chain**.

It does **not** perform:

ADC → FIR → FFT

Instead, it provides the **software/debug/communication interface**:

RISC-V ↔ UART ↔ Host PC

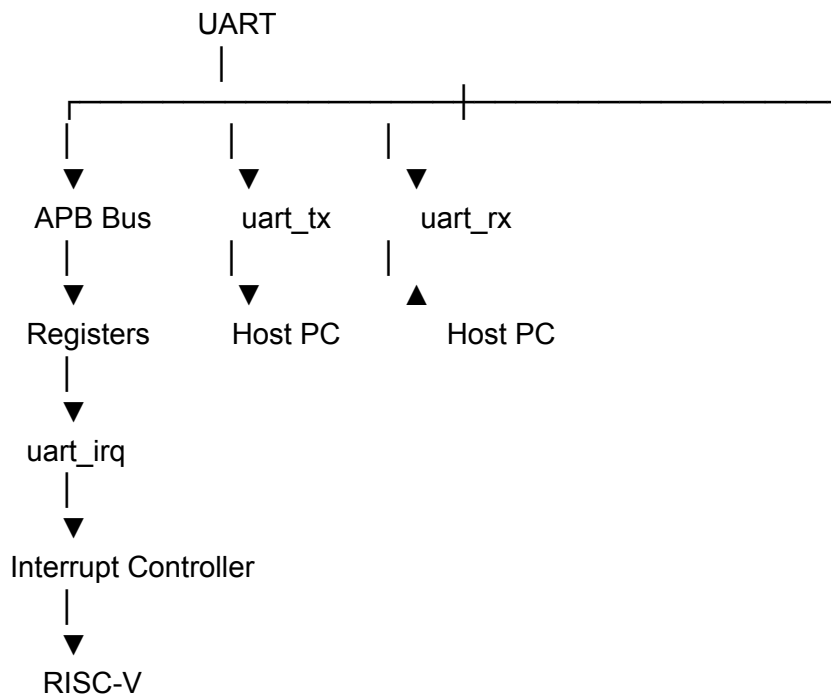
For example, your firmware could:

1. Configure ADC/FIR/FFT

2. Start spectrum acquisition
3. Wait for processing to complete
4. Read/format results
5. Send results to Host PC through UART

So UART makes the SoC's results **observable outside the chip**.

Final UART signal picture



For our detailed signal list, UART therefore has:

APB: PCLK, PRESETn, PADDR, PSEL, PENABLE, PWRITE, PWDATA, PRDATA, PREADY, PSLVERR

External: uart_tx, uart_rx

Interrupt: uart_irq

That's the complete **architecturally relevant UART signal set** for our current SoC specification.

8. Timer — Signal List

For our SoC, the Timer is an APB-controlled peripheral. Its main functions are **counting**, **programmable timeout/compare operation**, **status generation**, and **interrupt generation**.

APB Interface

Signal	Direction	Source → Destination	Width (bits)	Description
PADDR[31:0]	Input	APB Bus → Timer	32	Timer register address
PSEL	Input	APB Decoder → Timer	1	Selects Timer
PENABLE	Input	APB Bus → Timer	1	Indicates APB access phase
PWRITE	Input	APB Bus → Timer	1	Selects register read/write
PWDATA[31:0]	Input	APB Bus → Timer	32	Data written to Timer registers
PRDATA[31:0]	Output	Timer → APB Bus	32	Data read from Timer registers
PREADY	Output	Timer → APB Bus	1	Indicates completion of APB transfer
PSLVERR	Output	Timer → APB Bus	1	Indicates APB transfer error

Timer-Specific Signals

Signal	Direction	Source → Destination	Width (bits)	Description
TIMER_IRQ	Output	Timer → PIC/Interrupt Controller	1	Indicates timer interrupt/event
timer_count[31:0]	Internal	Timer counter logic	32	Current timer count value

<code>timer_compare[31:0]</code>	Internal	Timer register → Timer logic	32	Programmable value compared with counter
<code>timer_enable</code>	Internal	Control register → Timer logic	1	Enables timer counting
<code>timer_clear</code>	Internal	Control register → Timer logic	1	Clears/resets timer count
<code>timer_expired</code>	Internal	Timer logic → Status logic	1	Indicates compare/timeout event

Clock/Reset

`PCLK` and `PRESETn` are the common APB clock/reset signals already defined at the APB subsystem level, so **we don't create duplicate entries for them here**.

How it works

```

RISC-V
  ↓ APB write
Timer Control/Compare Registers
  ↓
Timer Counter
  ↓ compare match
TIMER_IRQ → PIC → RISC-V

```

The firmware can therefore configure the Timer through its APB registers and use `TIMER_IRQ` to notify RISC-V when the programmed timeout occurs.

Important: `timer_count`, `timer_compare`, `timer_enable`, etc. are **internal Timer signals**, not SoC-level interface signals. We'll retain them because your final document requires internal signals too, but later we'll clearly distinguish **internal signals from cross-block signals** so the same signal isn't documented twice.

9. GPIO — Signal List

GPIO is an APB-controlled peripheral that provides programmable **digital input and output pins**. In our SoC, it is mainly useful for simple external control/status signals and debugging.

APB Interface

Signal	Direction	Source → Destination	Width (bits)	Description
PADDR[31:0]	Input	APB Bus → GPIO	32	GPIO register address
PSEL	Input	APB Decoder → GPIO	1	Selects GPIO
PENABLE	Input	APB Bus → GPIO	1	Indicates APB access phase
PWRITE	Input	APB Bus → GPIO	1	Selects register read/write
PWDATA[31:0]	Input	APB Bus → GPIO	32	Data written to GPIO registers
PRDATA[31:0]	Output	GPIO → APB Bus	32	Data read from GPIO registers
PREADY	Output	GPIO → APB Bus	1	Indicates completion of APB transfer
PSLVERR	Output	GPIO → APB Bus	1	Indicates APB transfer error

GPIO-specific Signals

For our **8-bit GPIO configuration**:

Signal	Direction	Source → Destination	Width (bits)	Description
GPIO_IN[7:0]	Input	External → GPIO	8	External digital input signals

<code>GPIO_OUT[7:0]</code>	Output	GPIO → External	8	Digital output signals driven by GPIO
<code>GPIO_DIR[7:0]</code>	Internal/Register	GPIO control → GPIO logic	8	Configures each GPIO pin as input or output
<code>GPIO_IRQ</code>	Output	GPIO → PIC	1	Indicates a configured GPIO interrupt/event

Internal GPIO signals

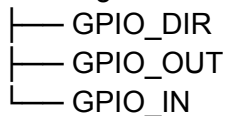
Signal	Direction	Source → Destination	Width	Description
<code>gpio_in_reg[7:0]</code>	Internal	GPIO input → GPIO logic	8	Registered input state
<code>gpio_out_reg[7:0]</code>	Internal	GPIO logic → GPIO output	8	Stored output value

Operation

RISC-V

↓ APB

GPIO Registers



↕

GPIO Logic

↕

External GPIO pins

If a GPIO interrupt is enabled and a configured input event occurs:

GPIO → GPIO_IRQ → PIC → RISC-V

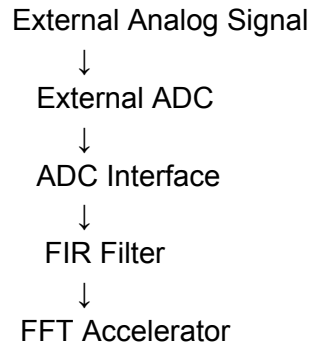
Important: `GPIO_IN[7:0]` and `GPIO_OUT[7:0]` are the actual signals that cross our SoC boundary. The APB and internal GPIO signals remain inside the SoC.

Block 9 complete. Next: Block 10 — ADC Interface, which is particularly important because it begins our actual **spectrum-analyzer DSP data path**.

10. ADC Interface — Signal List

This is the first block in our actual **DSP data-acquisition path**, so we need to distinguish its **control interface** from its **sample-data interface**.

Our architecture is:



The **External ADC** is **outside the SoC**; the **ADC Interface** is **inside the SoC**. The ADC Interface receives already-digitized samples and makes them available to the DSP chain.

A. APB Control Interface

Signal	Direction	Source → Destination	Width (bits)	Description
PADDR[31:0]	Input	APB Bus → ADC Interface	32	ADC interface register address
PSEL	Input	APB Decoder → ADC Interface	1	Selects ADC Interface
PENABLE	Input	APB Bus → ADC Interface	1	Indicates APB access phase
PWRITE	Input	APB Bus → ADC Interface	1	Selects register read/write
PWDATA[31:0]	Input	APB Bus → ADC Interface	32	Data written to ADC control registers
PRDATA[31:0]	Output	ADC Interface → APB Bus	32	Data read from ADC registers
PREADY	Output	ADC Interface → APB Bus	1	Indicates completion of APB transfer

PSLVERR	Output	ADC Interface → APB Bus	1	Indicates APB transfer error
---------	--------	-------------------------	---	------------------------------

B. External ADC → SoC

These are **boundary signals** because the physical ADC is external to our SoC.

Signal	Direction	Source → Destination	Width (bits)	Description
ADC_DATA[11:0]	Input	External ADC → ADC Interface	12	Digitized ADC sample
ADC_VALID	Input	External ADC → ADC Interface	1	Indicates that ADC_DATA contains a valid sample

C. ADC Interface → FIR

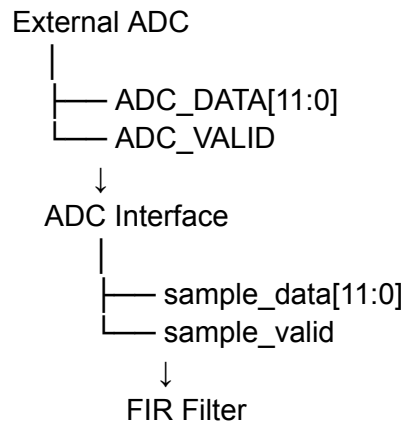
These are **cross-block DSP data-path signals**.

Signal	Direction	Source → Destination	Width (bits)	Description
sample_data[11:0]	Output	ADC Interface → FIR Filter	12	Valid digitized sample forwarded to FIR
sample_valid	Output	ADC Interface → FIR Filter	1	Indicates a valid sample is available

D. ADC Status / Interrupt

Signal	Direction	Source → Destination	Width (bits)	Description
ADC_IRQ	Output	ADC Interface → PIC	1	Indicates an ADC-related event requiring processor attention
sample_ready	Internal	ADC Interface → Control Logic	1	Indicates reception of a valid ADC sample

Operation



Meanwhile, RISC-V configures the ADC Interface through:

RISC-V → AHB → AHB-to-APB Bridge → APB → ADC Interface

Important clarification

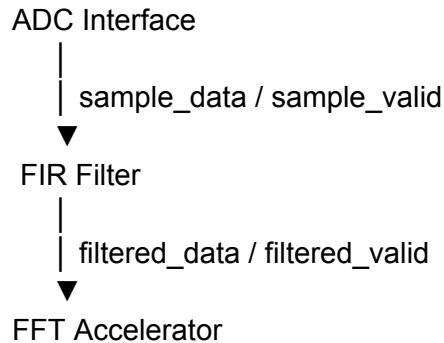
The **ADC Interface does not perform ADC conversion**. The external ADC has already converted the analog signal into digital samples. Our interface simply:

- receives the 12-bit samples,
- detects valid samples,
- optionally buffers/registers them,
- forwards them to the FIR filter,
- provides configuration/status to RISC-V.

And because we're keeping the signal documentation organized, **ADC_DATA/ADC_VALID are boundary signals**, while **sample_data/sample_valid are cross-block DSP signals**. We'll consolidate those later in the final cross-block/boundary sections rather than counting them twice.

11. FIR Filter — Signal List

This is the first actual **DSP processing IP** in our spectrum-analyzer chain. Its job is to take the digitized samples from the ADC Interface, apply the programmed FIR coefficients, and pass the filtered samples to the FFT Accelerator.



A. APB Control Interface

Signal	Direction	Source → Destination	Width (bits)	Description
<code>PADDR[31:0]</code>	Input	APB Bus → FIR	32	FIR register address
<code>PSEL</code>	Input	APB Decoder → FIR	1	Selects FIR Filter
<code>PENABLE</code>	Input	APB Bus → FIR	1	Indicates APB access phase
<code>PWRITE</code>	Input	APB Bus → FIR	1	Selects register read/write
<code>PWDATA[31:0]</code>	Input	APB Bus → FIR	32	Data written to FIR registers
<code>PRDATA[31:0]</code>	Output	FIR → APB Bus	32	Data read from FIR registers
<code>PREADY</code>	Output	FIR → APB Bus	1	Indicates completion of APB transfer
<code>PSLVERR</code>	Output	FIR → APB Bus	1	Indicates APB transfer error

B. Input Data Interface

Signal	Direction	Source → Destination	Width (bits)	Description
<code>sample_data[11:0]</code>	Input	ADC Interface → FIR	12	Digitized ADC sample
<code>sample_valid</code>	Input	ADC Interface → FIR	1	Indicates a valid input sample

C. Output Data Interface

Signal	Direction	Source → Destination	Width (bits)	Description
<code>filtered_data[15:0]</code>	Output	FIR → FFT	16	Filtered sample produced by FIR
<code>filtered_valid</code>	Output	FIR → FFT	1	Indicates valid filtered output

Why 16 bits? The input is 12-bit, but FIR multiplication and accumulation can require additional bits. We therefore allow a wider output to preserve precision. The **exact FIR coefficient width, accumulator width and final output width must be fixed when we define the FIR implementation**; `16-bit` here is our current architectural choice, not a VeeR/APB requirement.

D. FIR Control/Status

Signal	Direction	Source → Destination	Width (bits)	Description
<code>FIR_IRQ</code>	Output	FIR → PIC	1	Indicates a configured FIR event
<code>fir_enable</code>	Internal	Control register → FIR logic	1	Enables FIR processing
<code>fir_clear</code>	Internal	Control register → FIR logic	1	Clears FIR state/history
<code>fir_busy</code>	Internal	FIR logic → Status register	1	Indicates FIR is processing

<code>fir_done</code>	Internal	FIR logic → Status/IRQ logic	1	Indicates completion of configured processing
-----------------------	----------	------------------------------	---	---

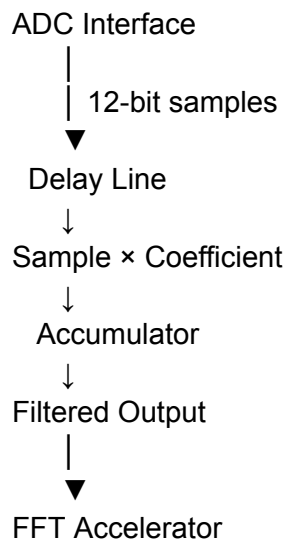
E. FIR Processing Internals

These are useful for the **detailed internal signal list**, because they represent the actual filtering operation.

Signal	Direction	Width	Description
<code>coeff[]</code>	Internal	TBD	Programmable FIR filter coefficients
<code>delay_line[]</code>	Internal	TBD	Stores previous input samples
<code>mult_result[]</code>	Internal	TBD	Results of sample × coefficient multiplication
<code>accumulator</code>	Internal	TBD	Accumulates multiplication results

The exact widths of these internal datapath signals should be finalized **after we decide the FIR tap count and coefficient representation**.

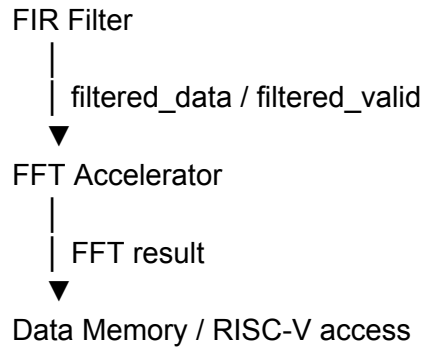
How FIR works in our SoC



RISC-V doesn't directly perform the filtering. It **configures the FIR through APB registers**, while the FIR hardware performs the multiply-accumulate operation.

12. FFT Accelerator — Signal List

This is the **main DSP acceleration IP** in our project. It receives the filtered samples from the FIR, computes the frequency-domain representation using FFT, and makes the resulting spectrum available to the RISC-V through its control/status interface.



A. APB Control Interface

Signal	Direction	Source → Destination	Width (bits)	Description
PADDR[31:0]	Input	APB Bus → FFT	32	FFT register address
PSEL	Input	APB Decoder → FFT	1	Selects FFT Accelerator
PENABLE	Input	APB Bus → FFT	1	Indicates APB access phase
PWRITE	Input	APB Bus → FFT	1	Selects register read/write
PWDATA[31:0]	Input	APB Bus → FFT	32	Data written to FFT registers
PRDATA[31:0]	Output	FFT → APB Bus	32	Data read from FFT registers
PREADY	Output	FFT → APB Bus	1	Indicates completion of APB transfer
PSLVERR	Output	FFT → APB Bus	1	Indicates APB transfer error

B. Input Data Interface

Signal	Direction	Source → Destination	Width (bits)	Description
<code>filtered_data[15:0]</code>	Input	FIR → FFT	16	Filtered time-domain sample
<code>filtered_valid</code>	Input	FIR → FFT	1	Indicates valid filtered sample

C. FFT Output / Data Path

We need to be careful here: **the exact output width and interface depend on how we implement the FFT accelerator**. For the specification, we can define the architectural signals as:

Signal	Direction	Source → Destination	Width (bits)	Description
<code>fft_real[31:0]</code>	Output	FFT → Data Storage	32	Real component of FFT output
<code>fft_imag[31:0]</code>	Output	FFT → Data Storage	32	Imaginary component of FFT output
<code>fft_valid</code>	Output	FFT → Data Storage	1	Indicates valid FFT output
<code>fft_done</code>	Internal/Status	FFT → Control Logic	1	Indicates completion of FFT processing

The **32-bit real/imaginary widths are a current architectural choice**, not something we should treat as fixed until we finalize the FFT implementation and numerical precision.

D. FFT Control / Status

Signal	Direction	Source → Destination	Width (bits)	Description
<code>FFT_IRQ</code>	Output	FFT → PIC	1	Indicates FFT completion/event

<code>fft_enable</code>	Internal	Control Register → FFT	1	Enables FFT operation
<code>fft_start</code>	Internal	Control Register → FFT	1	Starts FFT processing
<code>fft_reset</code>	Internal	Control Logic → FFT	1	Clears FFT processing state
<code>fft_busy</code>	Internal	FFT → Status Logic	1	Indicates FFT is processing
<code>fft_done</code>	Internal	FFT → Status Logic	1	Indicates FFT computation is complete

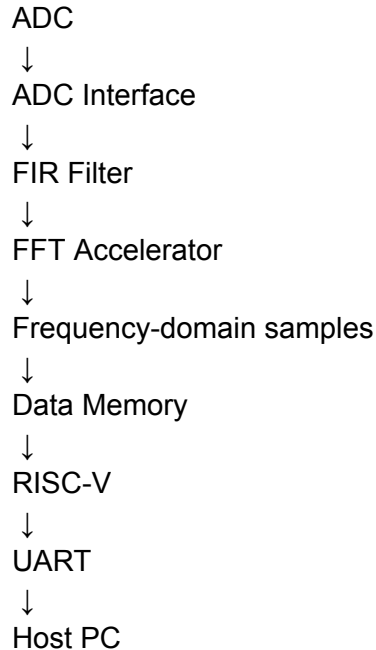
E. FFT Processing Internals

For the detailed internal signal list, we'll eventually have signals representing:

Signal	Width	Description
<code>fft_input_buffer[]</code>	TBD	Stores input samples for FFT
<code>fft_stage_data[]</code>	TBD	Intermediate data between FFT stages
<code>twiddle_factor[]</code>	TBD	Stores/represents FFT twiddle factors
<code>complex_mult_result</code>	TBD	Result of complex multiplication
<code>fft_output_buffer[]</code>	TBD	Stores final FFT results

The exact widths and number of elements depend on the **FFT size, architecture and fixed-point format**, which we should finalize before locking these into the specification.

How it works in our project



The important distinction is:

- **FIR** → cleans/filters the sampled signal.
- **FFT** → converts the filtered time-domain signal into its **frequency spectrum**.
- **RISC-V** → controls the accelerator and reads/processes the results.
- **UART** → allows the results/status to be sent to the Host PC.

One correction we should keep in our final architecture

We should **not yet claim a direct FFT → Data Memory interface with specific signal names** unless we decide that the FFT is actually a memory-writing master.

If the FFT is instead accessed through registers/FIFO by RISC-V, the architecture is different.

So for now, `fft_real`, `fft_imag`, and `fft_valid` represent the **DSP output interface**, while the exact **FFT → memory mechanism** will be finalized when we design the data path.

Block 13 — VeeR EL2 ↔ AHB-Lite Interconnect

VeeR has two AHB-Lite interfaces: **Instruction Fetch (IFU)** and **Load/Store (LSU)**. The IFU is used for instruction fetches, while the LSU handles data and memory-mapped peripheral accesses.

A. Instruction Fetch Interface

Signal	Direction	Source → Destination	Width (bits)	Description
HADDR[31:0]	Output	VeeR IFU → AHB Interconnect	32	Instruction fetch address
HTRANS[1:0]	Output	VeeR IFU → AHB Interconnect	2	AHB transfer type
HSIZE[2:0]	Output	VeeR IFU → AHB Interconnect	3	Transfer size
HBURST[2:0]	Output	VeeR IFU → AHB Interconnect	3	Burst type
HPROT[3:0]	Output	VeeR IFU → AHB Interconnect	4	Transfer protection attributes
HWRITE	Output	VeeR IFU → AHB Interconnect	1	Read/write indication

HMASTLOCK	Output	VeeR IFU → AHB Interconnect	1	Locked-transfer indication
HRDATA[63:0]	Input	AHB Interconnect → VeeR IFU	64	Instruction data returned
HREADY	Input	AHB Interconnect → VeeR IFU	1	Indicates transfer completion
HRESP	Input	AHB Interconnect → VeeR IFU	1	Indicates transfer response/error

B. Load/Store Interface

Signal	Direction	Source → Destination	Width (bits)	Description
HADDR[31:0]	Output	VeeR LSU → AHB Interconnect	32	Data/peripheral access address
HTRANS[1:0]	Output	VeeR LSU → AHB Interconnect	2	AHB transfer type
HSIZE[2:0]	Output	VeeR LSU → AHB Interconnect	3	Transfer size

HBURST[2:0]	Output	VeeR LSU → AHB Interconnect	3	Burst type
HPROT[3:0]	Output	VeeR LSU → AHB Interconnect	4	Transfer protection attributes
HWRITE	Output	VeeR LSU → AHB Interconnect	1	Read/write indication
HMASTLOCK	Output	VeeR LSU → AHB Interconnect	1	Locked-transfer indication
HWDATA[63:0]	Output	VeeR LSU → AHB Interconnect	64	Write data
HRDATA[63:0]	Input	AHB Interconnect → VeeR LSU	64	Read data
HREADY	Input	AHB Interconnect → VeeR LSU	1	Indicates transfer completion
HRESP	Input	AHB Interconnect → VeeR LSU	1	Indicates transfer response/error

These widths are based on the VeeR EL2 documented IFU/LSU AHB-Lite interfaces.

Important naming note

For the **architecture document**, we'll use standard AHB names:

HADDR, HTRANS, HSIZE, HBURST, HPROT, HWRITE, HWDATA, HRDATA, HREADY, HRESP, HMASTLOCK

When we later prepare the **actual RTL-level signal list**, we'll preserve VeeR's actual port names such as `lsu_haddr`, `lsu_hwdata`, etc.

Cross-block flow

VeeR IFU — AHB request —→ AHB Interconnect

VeeR LSU — AHB request —→ AHB Interconnect

AHB Interconnect — response/data —→ VeeR IFU

AHB Interconnect — response/data —→ VeeR LSU

Block 14 — AHB-Lite ↔ Instruction/Data Memory

This covers the two memory regions in our SoC:

- **Instruction Memory** → accessed primarily by the VeeR IFU for instruction fetch.
- **Data Memory** → accessed by the VeeR LSU for data and DSP-related storage.

We will document the **AHB-side signals crossing into the memories**. Memory-internal signals will be handled separately.

A. AHB → Instruction Memory

Signal	Direction	Source → Destination	Width (bits)	Description
HADDR[31:0]	Input	AHB Interconnect → Instruction Memory	32	Address of instruction being fetched
HTRANS[1:0]	Input	AHB Interconnect → Instruction Memory	2	AHB transfer type
HSIZE[2:0]	Input	AHB Interconnect → Instruction Memory	3	Size of requested transfer
HBURST[2:0]	Input	AHB Interconnect → Instruction Memory	3	Burst information
HPROT[3:0]	Input	AHB Interconnect → Instruction Memory	4	Transfer protection attributes
HWRITE	Input	AHB Interconnect → Instruction Memory	1	Indicates read/write operation

HMASTLOCK	Input	AHB Interconnect → Instruction Memory	1	Indicates locked transfer
HREADY	Input	AHB Interconnect → Instruction Memory	1	Indicates transfer availability/completion

Instruction Memory → AHB

Signal	Direction	Source → Destination	Width (bits)	Description
HRDATA[63:0]	Output	Instruction Memory → AHB Interconnect	64	Instruction data returned
HREADYOUT	Output	Instruction Memory → AHB Interconnect	1	Indicates memory transfer completion
HRESP	Output	Instruction Memory → AHB Interconnect	1	Indicates memory access response

B. AHB → Data Memory

The Data Memory uses the same AHB control/address interface, but additionally supports write data because the LSU can both **read and write** data.

Signal	Direction	Source → Destination	Width (bits)	Description
--------	-----------	----------------------	--------------	-------------

HADDR[31:0]	Input	AHB Interconnect → Data Memory	32	Data-memory address
HTRANS[1:0]	Input	AHB Interconnect → Data Memory	2	AHB transfer type
HSIZE[2:0]	Input	AHB Interconnect → Data Memory	3	Size of requested transfer
HBURST[2:0]	Input	AHB Interconnect → Data Memory	3	Burst information
HPROT[3:0]	Input	AHB Interconnect → Data Memory	4	Transfer protection attributes
HWRITE	Input	AHB Interconnect → Data Memory	1	Indicates read/write operation
HWDATA[63:0]	Input	AHB Interconnect → Data Memory	64	Data written to memory
HMASTLOCK	Input	AHB Interconnect → Data Memory	1	Indicates locked transfer
HREADY	Input	AHB Interconnect → Data Memory	1	Indicates transfer availability/completion

Data Memory → AHB

Signal	Direction	Source → Destination	Width (bits)	Description
HRDATA[63:0]	Output	Data Memory → AHB Interconnect	64	Data read from memory
HREADYOUT	Output	Data Memory → AHB Interconnect	1	Indicates memory transfer completion
HRESP	Output	Data Memory → AHB Interconnect	1	Indicates memory access response

Memory access flow

VeeR IFU → AHB Interconnect → Instruction Memory

Instruction Memory → AHB Interconnect → VeeR IFU

VeeR LSU → AHB Interconnect → Data Memory

Data Memory → AHB Interconnect → VeeR LSU

Important

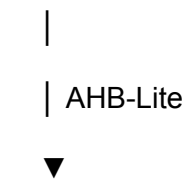
HADDR, HWRITE, HWDATA, etc. appear in both memory interfaces because **they are the same AHB protocol signals used for different AHB targets**. They are not separate physical signal definitions in the overall SoC; the interconnect routes them to the selected memory.

Also, HREADYOUT is **memory-side terminology**. At the interconnect/master-facing side it contributes to the HREADY response. We'll keep this distinction when we later make the complete RTL-level signal list.

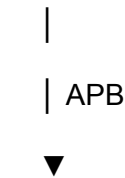
Block 15 — Signal List

This is the **protocol conversion point** of our SoC. The VeeR side communicates using AHB-Lite, while the peripheral subsystem uses APB. The bridge converts an AHB transaction into an APB transaction and converts the APB response back to AHB.

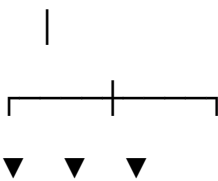
VeeR / AHB Interconnect



AHB-to-APB Bridge



APB Decoder



UART Timer GPIO ...

A. AHB-Lite Side

AHB → Bridge

Signal	Direction	Source → Destination	Width (bits)	Description
HADDR[31:0]	Input	AHB Interconnect → Bridge	32	Address of peripheral access

HTRANS[1:0]	Input	AHB Interconnect → Bridge	2	AHB transfer type
HSIZE[2:0]	Input	AHB Interconnect → Bridge	3	Transfer size
HBURST[2:0]	Input	AHB Interconnect → Bridge	3	Burst information
HPROT[3:0]	Input	AHB Interconnect → Bridge	4	Transfer protection attributes
HWRITE	Input	AHB Interconnect → Bridge	1	Indicates read/write operation
HWDATA[63:0]	Input	AHB Interconnect → Bridge	64	Write data
HMASTLOCK	Input	AHB Interconnect → Bridge	1	Locked-transfer indication

Bridge → AHB

Signal	Direction	Source → Destination	Width (bits)	Description
HRDATA[63:0]	Output	Bridge → AHB Interconnect	64	Read data returned from APB peripheral

HREADY	Output	Bridge → AHB Interconnect	1	Indicates completion of AHB transaction
HRESP	Output	Bridge → AHB Interconnect	1	Indicates AHB transfer response/error

B. APB Side

Bridge → APB

Signal	Direction	Source → Destination	Width (bits)	Description
PADDR[31:0]	Output	Bridge → APB Decoder	32	Converted APB peripheral address
PWRITE	Output	Bridge → APB Decoder	1	Indicates APB read/write operation
PWDATA[31:0]	Output	Bridge → APB Decoder	32	Write data for APB peripheral
PSEL	Output	Bridge → APB Decoder	1	Indicates an active APB transaction
PENABLE	Output	Bridge → APB Decoder	1	Indicates APB ACCESS phase

APB → Bridge

Signal	Direction	Source → Destination	Width (bits)	Description
PRDATA[31:0]	Input	APB Decoder → Bridge	32	Read data returned by peripheral
PREADY	Input	APB Decoder → Bridge	1	Indicates APB transaction completion
PSLVERR	Input	APB Decoder → Bridge	1	Indicates APB transfer error

C. Clock and Reset

HCLK/system clock and reset are **common SoC-level signals**, so we don't redefine them as new functional data-path signals here. The bridge operates synchronously with the SoC clock.

How the bridge works

For a **write**:

AHB Write



AHB-to-APB Bridge



APB SETUP



APB ACCESS



Selected Peripheral

For a **read**:

Selected Peripheral



APB



AHB-to-APB Bridge



AHB Interconnect



RISC-V

The key purpose is therefore:

AHB-Lite transaction → APB transaction → peripheral access → APB response → AHB-Lite response.

Important width point

Notice that our AHB data path is **64-bit**, while APB is **32-bit**. Therefore, the bridge must handle the required **data-width conversion** when moving between the two interfaces.

Also, **HBURST/HMASTLOCK** are shown because they belong to the AHB-side interface, but our actual APB peripheral accesses will normally be treated as **individual APB transfers**, not APB bursts.

16. APB Bus/Decoder ↔ Peripherals

This section captures the **common APB connections from the APB decoder to all six peripherals**. Since the APB protocol signals are identical for each peripheral, we should **not create six repetitive tables**. Instead, we define the common signals once and then list the individual **PSEL** signals.

A. Common APB Signals

Signal	Direction	Source → Destination	Width (bits)	Description
PADDR[31:0]	Output	APB Decoder → All APB Peripherals	32	Address of the selected peripheral register
PWDATA[31:0]	Output	APB Decoder → All APB Peripherals	32	Write data
PWRITE	Output	APB Decoder → All APB Peripherals	1	Indicates read (0) or write (1)
PENABLE	Output	APB Decoder → All APB Peripherals	1	Indicates APB ACCESS phase
PCLK	Input	SoC → All APB Peripherals	1	APB clock
PRESETn	Input	SoC → All APB Peripherals	1	Active-low APB reset

For a given transaction, these signals are distributed to the APB peripherals, while only the **selected peripheral** responds.

B. Peripheral Select Signals

Signal	Direction	Source → Destination	Width (bits)	Description
PSEL_UART	Output	APB Decoder → UART	1	Selects UART
PSEL_TIMER	Output	APB Decoder → Timer	1	Selects Timer
PSEL_GPIO	Output	APB Decoder → GPIO	1	Selects GPIO
PSEL_ADC	Output	APB Decoder → ADC Interface	1	Selects ADC Interface
PSEL_FILTER	Output	APB Decoder → FIR Filter	1	Selects FIR Filter
PSEL_FFT	Output	APB Decoder → FFT Accelerator	1	Selects FFT Accelerator

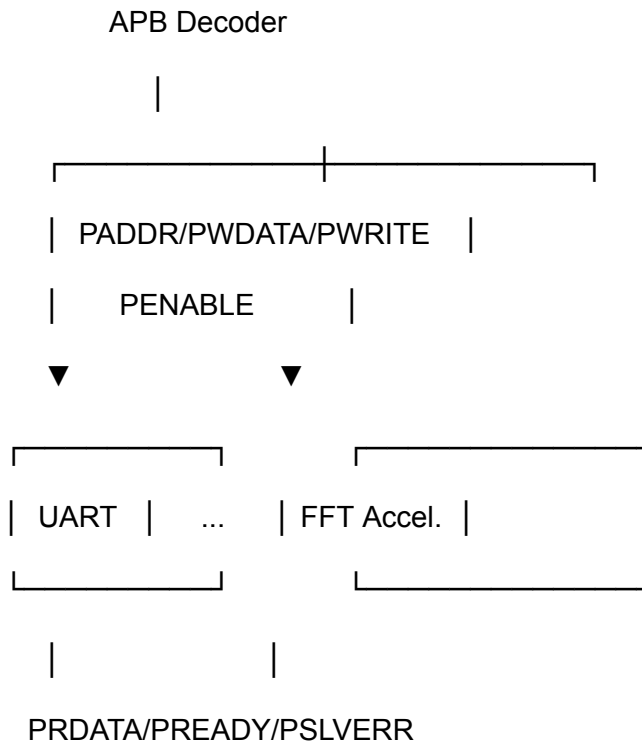
C. Peripheral Response Signals

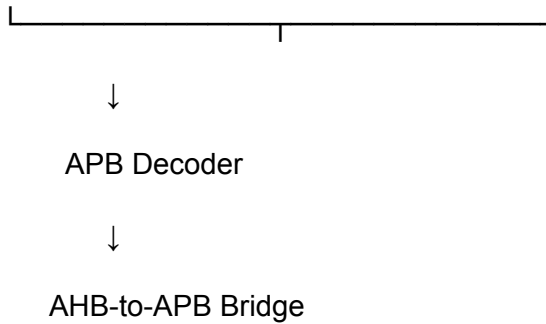
Each peripheral returns the standard APB response:

Signal	Direction	Source → Destination	Width (bits)	Description
PRDATA[31:0]	Input	Selected Peripheral → APB Decoder	32	Read data
PREADY	Input	Selected Peripheral → APB Decoder	1	Indicates transfer completion
PSLVERR	Input	Selected Peripheral → APB Decoder	1	Indicates APB transfer error

The decoder/multiplexer selects the response from the currently addressed peripheral and sends it back toward the **AHB-to-APB bridge**.

Overall connection





Key point for the specification

APB Decoder → **peripherals**: common APB signals + individual **PSEL**.

Peripherals → **APB Decoder**: **PRDATA**, **PREADY**, **PSLVERR**.

The **peripheral-specific signals** such as **UART_TX**, **ADC_DATA**, **sample_valid**, **filtered_data**, etc. are **not included here** because we already document them under their respective blocks/cross-block connections.

17. DSP DATA PATH

17A. ADC Interface ↔ FIR Filter

This is the **first direct DSP data-path connection**. The ADC Interface receives the digitized samples from the external ADC and forwards valid samples to the FIR Filter.

Signal	Direction	Source → Destination	Width (bits)	Description
<code>sample_data[11:0]</code>	Output	ADC Interface → FIR Filter	12	Digitized ADC sample forwarded for filtering
<code>sample_valid</code>	Output	ADC Interface → FIR Filter	1	Indicates that <code>sample_data</code> contains a valid sample

Data flow

External ADC



ADC Interface



|— `sample_data[11:0]` —→ FIR Filter

|— `sample_valid` —→ FIR Filter

The FIR processes `sample_data` **only when** `sample_valid = 1`. This keeps the processing synchronized with the incoming ADC samples.

Important

There is **no separate clock/reset signal listed here** because clock and reset are common SoC signals already defined at the system/interface level.

Also, `sample_data` and `sample_valid` are **not new signals created at this connection**—they are the same cross-block signals already identified from the ADC Interface's output side. In the final consolidated signal list, they should appear **only once**.

17B. FIR Filter ↔ FFT Accelerator

This is the next DSP connection:

Signal	Direction	Source → Destination	Width (bits)	Description
<code>filtered_data[15:0]</code>	Output	FIR Filter → FFT Accelerator	16	Filtered time-domain sample provided to FFT
<code>filtered_valid</code>	Output	FIR Filter → FFT Accelerator	1	Indicates that <code>filtered_data</code> contains a valid sample

Data Flow

ADC Interface

```
|
| sample_data[11:0]
| sample_valid
```



FIR Filter

```
|
| filtered_data[15:0]
| filtered_valid
```



FFT Accelerator

Purpose: The FIR output becomes the input to the FFT. `filtered_valid` ensures the FFT accepts the sample only when valid.

17C. FFT Accelerator ↔ Data Memory

However, there is one architectural point we should lock down: **our FFT accelerator should produce the frequency-domain results and store them in Data Memory so that RISC-V can later access/process/transmit them.** For this architecture, we'll therefore define a dedicated FFT-to-memory write interface.

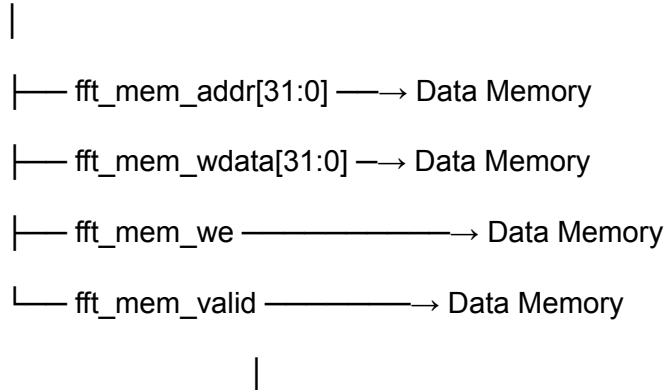
Signal	Direction	Source → Destination	Width (bits)	Description
<code>fft_mem_addr[31:0]</code>	Output	FFT Accelerator → Data Memory	32	Address where FFT result is stored
<code>fft_mem_wdata[31:0]</code>	Output	FFT Accelerator → Data Memory	32	FFT result data written to memory
<code>fft_mem_we</code>	Output	FFT Accelerator → Data Memory	1	Enables FFT result write to memory
<code>fft_mem_valid</code>	Output	FFT Accelerator → Data Memory	1	Indicates a valid memory-write request
<code>fft_mem_ready</code>	Input	Data Memory → FFT Accelerator	1	Indicates memory can accept the write

Data Flow

FIR Filter



FFT Accelerator



fft_mem_ready



FFT Accelerator

What happens here?

1. FFT processes the filtered samples.
2. It generates the **frequency-domain result**.
3. FFT generates a memory address and result data.
4. `fft_mem_valid` + `fft_mem_we` indicate a write request.
5. Data Memory accepts the result when `fft_mem_ready = 1`.
6. RISC-V can subsequently access the stored spectrum through the normal AHB-Lite data path.

One important specification point

The **32-bit `fft_mem_wdata`** is our current architectural choice. Since an FFT result is complex, we need to decide whether one 32-bit word represents:

- real component,
- imaginary component, or
- packed real + imaginary components.

We should finalize this when we define the FFT numerical format and memory/register specification. We shouldn't leave that ambiguous in the final specification document.

So 17A, 17B and 17C together completely describe our primary DSP data path.

18. Clock / Reset

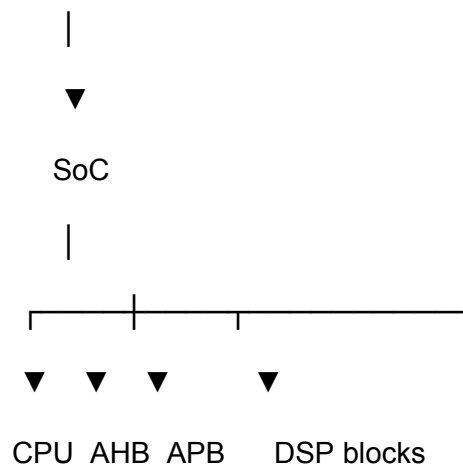
Yes. Now we move to **Block 18 — Boundary Signals: Clock / Reset**.

These are **global SoC boundary signals**, not part of the DSP data path. They provide the timing reference and initialize the SoC into a known state.

Signal	Direction	Source → Destination	Width (bits)	Description
clk	Input	External clock source → SoC	1	Main system clock for SoC operation
reset	Input	External reset source → SoC	1	Resets SoC logic to a known initial state

Clock distribution

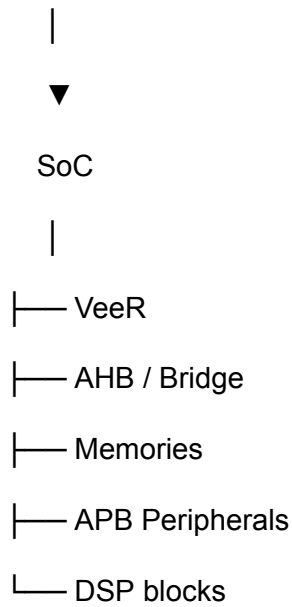
External Clock



The same system clock is distributed to the processor, buses, memories, peripherals and DSP logic as required.

Reset distribution

External Reset



Reset brings the SoC components into their defined initial states.

Important naming point

At this **SoC boundary level**, `clk` and `reset` are the clean architectural names. Individual RTL blocks may use names such as `clk_i`, `rst_ni`, `resetn`, etc., depending on the IP implementation. We should **not introduce those as separate signals** unless the actual RTL requires them.

19. External ADC ↔ SoC

Signal	Direction	Source → Destination	Width (bits)	Description
ADC_DATA[11:0]	Input	External ADC → SoC / ADC Interface	12	Digitized ADC sample provided to the SoC
ADC_VALID	Input	External ADC → SoC / ADC Interface	1	Indicates that ADC_DATA contains a valid sample

These are the **only ADC signals crossing the SoC boundary** in our current architecture. The signals `sample_data` and `sample_valid` are **internal cross-block signals** between the ADC Interface and FIR Filter, so they do **not** belong in Block 19.

20. Host PC ↔ UART

The Host PC communicates with the SoC through the UART peripheral. UART provides the external communication interface for sending configuration, status, and processed spectrum/results between the SoC and the Host PC.

Signal	Direction	Source → Destination	Width (bits)	Description
UART_RX	Input	Host PC → SoC / UART	1	Serial data received by the SoC
UART_TX	Output	SoC / UART → Host PC	1	Serial data transmitted from the SoC

These are the **only UART signals crossing the SoC boundary**. Signals such as PADDR, PWDATA, PRDATA, etc. belong to the **internal APB ↔ UART interface** and are therefore not repeated here.

21. GPIO ↔ External

The GPIO peripheral provides general-purpose digital input and output signals between the SoC and external devices.

Signal	Direction	Source → Destination	Width (bits)	Description
<code>GPIO_IN[7:0]</code>	Input	External → SoC / GPIO	8	8-bit external digital input received by GPIO
<code>GPIO_OUT[7:0]</code>	Output	SoC / GPIO → External	8	8-bit digital output driven by GPIO

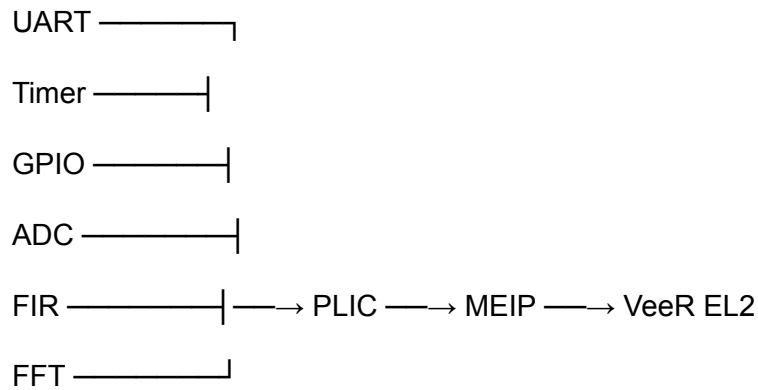
These are the **complete GPIO boundary signals** in our current architecture. The GPIO configuration/control signals (`PADDR`, `PWDATA`, `PRDATA`, `PWRITE`, `PSEL`, `PENABLE`, etc.) are **internal APB signals** and therefore are not included in the boundary list.

22. Peripheral → RISC-V Interrupt Paths

The interrupt system allows peripherals to notify the RISC-V processor when an important event occurs. The individual peripheral interrupt outputs are collected by the **Platform-Level Interrupt Controller (PLIC)** and then delivered to the RISC-V core.

Signal	Direction	Source → Destination	Width (bits)	Description
UART_IRQ	Output	UART → PLIC	1	Indicates a UART event requiring processor attention
TIMER_IRQ	Output	Timer → PLIC	1	Indicates timer compare/timeout event
GPIO_IRQ	Output	GPIO → PLIC	1	Indicates a configured GPIO event
ADC_IRQ	Output	ADC Interface → PLIC	1	Indicates an ADC-related event, such as sample availability
FIR_IRQ	Output	FIR Filter → PLIC	1	Indicates FIR processing/event completion
FFT_IRQ	Output	FFT Accelerator → PLIC	1	Indicates FFT processing completion/result availability
MEIP	Input	PLIC → VeeR EL2	1	Machine external interrupt delivered to RISC-V

Interrupt Path



Interrupt operation

1. A peripheral generates its interrupt request when its configured event occurs.
2. The PLIC receives and manages the interrupt requests.
3. The PLIC selects the appropriate pending interrupt according to its priority/enable configuration.
4. **MEIP** is asserted toward the VeeR processor.
5. RISC-V enters its interrupt handler and services the corresponding peripheral.
6. The interrupt source is cleared, allowing subsequent interrupts.

Important: **MEIP** is the **processor-facing interrupt signal**. The individual ***_IRQ** signals are the **peripheral-to-PLIC signals**.

23. NON-FATAL ERRORS + RECOVERY

Non-fatal errors are errors that do not require the complete SoC to be reset or powered down. The affected transaction or peripheral operation can be terminated or retried while the rest of the SoC continues operating.

Error Signal / Condition	Source → Destination	Width (bits)	Description	Recovery
HRESP	AHB Slave → AHB Interconnect	1	Indicates an AHB transfer error	Abort/terminate the affected transaction; processor can handle the exception
PSLVERR	APB Peripheral → APB Bridge	1	Indicates an APB peripheral access error	Return error to AHB side; software can retry or handle the failed access
PREADY timeout	APB Bridge ↔ Peripheral	1	Peripheral does not complete the APB transfer within the expected time	Abort/recover the affected transaction
Invalid peripheral address	AHB Interconnect → Bridge	—	Access to an unmapped/unsupported peripheral address	Generate bus error; software handles the exception
FIR processing error/status	FIR → RISC-V	1	Indicates an abnormal FIR operation, if implemented	Clear/reconfigure FIR and restart operation

FFT processing error/status	FFT → RISC-V	1	Indicates an abnormal FFT operation, if implemented	Clear/reconfigure FFT and restart operation
-----------------------------	--------------	---	---	---

Recovery Path

Error detected



Identify affected transaction/IP



Terminate or clear affected operation



Report error to RISC-V



Software handles error



Retry / reconfigure affected IP



Normal operation resumes

Key Point

A non-fatal error affects only the associated transaction or peripheral operation. It does not require a global SoC reset or power-off.

24. FATAL ERRORS + RESET / RECOVERY

Fatal errors are severe system-level failures that can leave the SoC in an unsafe or undefined operating state. These errors require **global recovery**, typically through a system reset; in a real hardware system, persistent critical faults may additionally require power cycling.

Fatal Error / Condition	Source → Destination	Width (bits)	Description	Recovery
System clock failure	Clock source → SoC	1	Loss or failure of the system clock	External reset / power cycle
Global reset failure	Reset source → SoC	1	Reset cannot correctly initialize the system	External intervention / power cycle
Severe bus/interconnect failure	AHB Interconnect → System	1	Critical bus failure preventing normal system operation	System reset
Unrecoverable memory failure	Memory → System	1	Critical memory fault that cannot be handled by normal software recovery	System reset
Unrecoverable processor failure	VeeR EL2 → System	1	Processor enters an unrecoverable state	System reset

Fatal Error Recovery Path

Fatal error detected



Stop normal operation



Assert system reset



Reset CPU + memories + buses + peripherals + DSP IPs



Reinitialize system



Resume normal operation

If the system cannot recover through reset:

Fatal / persistent hardware failure



System shutdown



Power-off



Power-cycle



System restart

Key Point

Non-fatal error → local recovery.

Fatal error → global reset.

Persistent unrecoverable failure → power cycle.

CONTROL PATH

1. Purpose

The control path defines how the **VeeR EL2 RISC-V processor configures, starts, monitors, and controls the peripherals and DSP accelerators in the SoC**. It also defines how status information, completion events, interrupts, and error information are returned to the processor.

The control path is separate from the DSP data path. The control path carries **configuration, commands, status, and control information**, while the DSP data path carries the actual sampled signal and processed spectrum data.

The overall control hierarchy is:

VeeR EL2 RISC-V

|

| AHB-Lite



AHB-Lite Interconnect

|

| Peripheral address region



AHB-to-APB Bridge

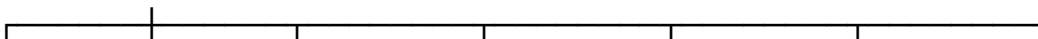
|

| APB



APB Decoder / Bus

|



UART Timer GPIO ADC FIR FFT

The RISC-V processor is therefore the **software-visible control master**, while the peripherals and DSP IPs are **memory-mapped controlled devices**.

2. Control Path Architecture

The control path uses the following sequence for accesses to APB peripherals:

RISC-V

|

| Memory-mapped read/write

▼

AHB-Lite Interconnect

|

| AHB transaction

▼

AHB-to-APB Bridge

|

| APB transaction

▼

APB Decoder

|

| PSEL for selected peripheral

▼

Peripheral Register Bank

For a read transaction, the response follows the reverse direction:

Peripheral Register

|

| PRDATA



APB Decoder

|



AHB-to-APB Bridge

|



AHB-Lite Interconnect

|



RISC-V

For a write transaction:

RISC-V

|

| Address + Write Data



AHB-Lite



AHB-to-APB Bridge



APB



Selected Peripheral Register

The bridge is responsible for converting the AHB-Lite transaction into the corresponding APB transaction.

3. Memory-Mapped Control

All APB peripherals are assigned fixed memory-mapped address regions. The RISC-V accesses these addresses using normal load/store instructions.

The current peripheral address allocation is:

Peripheral	Base Address
UART	0x4000_0000
Timer	0x4000_1000
GPIO	0x4000_2000
ADC Interface	0x4000_3000
FIR Filter	0x4000_4000
FFT Accelerator	0x4000_5000

The address presented by the RISC-V determines which peripheral is selected by the AHB/APB infrastructure.

For example:

RISC-V accesses 0x4000_4000



AHB transaction



AHB-to-APB Bridge



APB address = 0x4000_4000



PSEL_FIR = 1



FIR register selected

The exact register offset determines the specific register within that peripheral.

4. Peripheral Register Access

Each peripheral exposes a set of control and status registers through APB.

A typical register write follows:

RISC-V



Write instruction



AHB-Lite transaction



AHB-to-APB Bridge



APB SETUP phase



APB ACCESS phase



Peripheral register updated

A register read follows:

RISC-V



Read instruction



AHB-Lite transaction



AHB-to-APB Bridge



APB SETUP



APB ACCESS



Peripheral provides PRDATA



Bridge converts response to AHB



RISC-V receives data

The processor uses these accesses to configure the system before starting signal processing and to monitor the system during operation.

5. System Initialization and Control Sequence

After reset, the RISC-V begins execution from the configured instruction memory.

The software initialization sequence should be:

System Reset



RISC-V initialization



Initialize UART



Initialize Timer/GPIO as required



Initialize ADC Interface



Configure FIR



Configure FFT



Enable required interrupts



Start acquisition/processing

The exact initialization order can be modified during implementation, but all required configuration must be completed before normal spectrum-analysis processing begins.

6. UART Control

The UART provides communication between the SoC and the Host PC.

The RISC-V configures the UART using its memory-mapped registers.

Typical configuration includes:

- UART enable
- Baud-rate configuration
- TX/RX control
- Interrupt enable, if required

The control flow is:

RISC-V

↓

UART_CTRL

UART_BAUD

UART_IRQ_EN

↓

UART configured

↓

UART ↔ Host PC

The RISC-V can read **UART_STATUS** to determine whether transmit or receive operations are ready or complete.

UART is primarily used for **external communication and result transmission**, rather than for the DSP computation itself.

7. Timer Control

The Timer is controlled through its memory-mapped registers.

The RISC-V can configure:

TIMER_CTRL

TIMER_COUNT

TIMER_COMPARE

TIMER_IRQ_EN

A typical sequence is:

RISC-V



Configure TIMER_COUNT



Configure TIMER_COMPARE



Enable Timer



Timer counts



Compare/timeout event



TIMER_IRQ



PLIC



RISC-V

The Timer can therefore provide timing events to software when required by the system implementation.

8. GPIO Control

GPIO is configured by the RISC-V through its APB registers.

The processor can configure the direction and state of the GPIO pins.

RISC-V



GPIO_DIR



Configure GPIO direction



GPIO_DATA / GPIO_SET / GPIO_CLEAR



GPIO operation

If GPIO interrupt functionality is implemented and enabled:

External GPIO event



GPIO_IRQ



PLIC

↓

RISC-V

GPIO is a general-purpose control/status interface and is not part of the primary DSP data path.

9. ADC Interface Control

The external ADC performs the actual analog-to-digital conversion. The SoC does not perform the analog conversion itself.

The ADC Interface receives:

ADC_DATA[11:0]

ADC_VALID

from the external ADC.

The RISC-V configures the internal ADC Interface through its registers:

ADC_CTRL

ADC_STATUS

ADC_IRQ_EN

The control sequence is:

RISC-V

↓

Configure ADC Interface

↓

Enable acquisition

↓

External ADC supplies digital samples

↓

ADC Interface receives ADC_DATA



ADC_VALID indicates valid sample



Sample forwarded to FIR

The ADC Interface therefore acts as the boundary between the external ADC and the internal DSP processing pipeline.

10. FIR Filter Control

The FIR Filter is a custom DSP IP in the SoC.

Before filtering begins, the RISC-V must configure the FIR Filter and load the required coefficients.

The FIR control registers include:

FIR_CTRL

FIR_STATUS

FIR_COEFF_ADDR

FIR_COEFF_DATA

FIR_IRQ_EN

FIR Configuration Sequence

The processor first selects a coefficient location:

RISC-V



FIR_COEFF_ADDR

and then writes the corresponding coefficient:

RISC-V



FIR_COEFF_DATA

This process is repeated until all required FIR coefficients have been programmed.

After coefficient configuration:

FIR_CTRL



Enable FIR



FIR ready for sample processing

Once enabled, the FIR receives samples from the ADC Interface through:

sample_data[11:0]

sample_valid

The FIR performs the filtering operation in hardware.

The processor does **not** need to perform the multiply-accumulate operation sample by sample.

11. FFT Accelerator Control

The FFT Accelerator is the second custom DSP IP.

Before FFT processing begins, the RISC-V configures the accelerator through its memory-mapped registers.

The current proposed registers are:

FFT_CTRL

FFT_STATUS

FFT_SIZE

FFT_DATA

FFT_OUT_ADDR

FFT_IRQ_EN

The configuration sequence is:

RISC-V



Configure FFT_SIZE



Configure FFT control parameters



Enable FFT



Start FFT



FFT waits for/receives filtered samples



FFT processing



FFT completion

The FFT receives filtered samples from the FIR:

FIR



└─ filtered_data[15:0]

└─ filtered_valid



FFT Accelerator

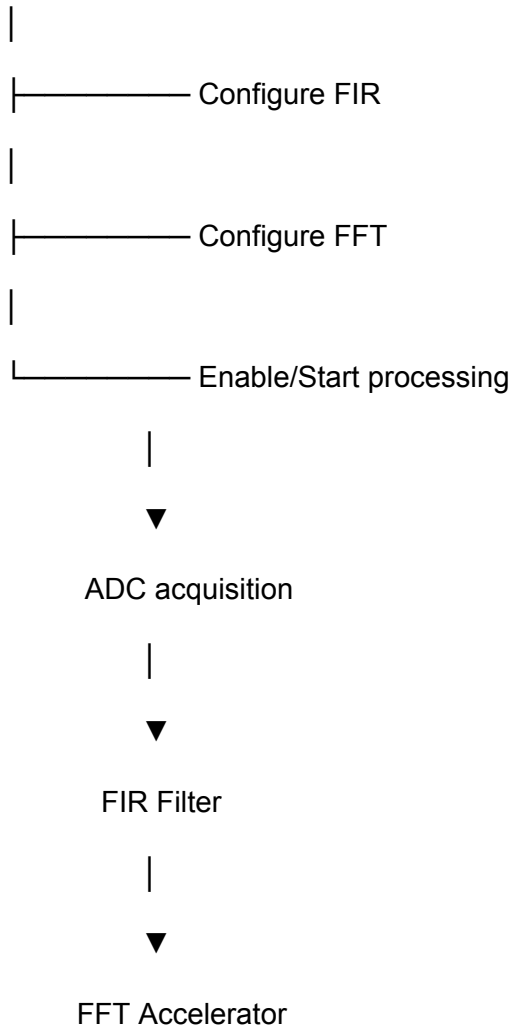
The FFT converts the time-domain samples into their frequency-domain representation.

12. DSP Accelerator Coordination

The FIR and FFT must operate in the correct sequence.

The intended control relationship is:

RISC-V





Data Memory

The processor therefore **controls the processing stages**, while the actual sample processing occurs in hardware.

13. Status Monitoring

Each major peripheral/DSP IP provides status information through its status registers.

The processor can read:

UART_STATUS

TIMER_STATUS

ADC_STATUS

FIR_STATUS

FFT_STATUS

to determine the state of the corresponding block.

For the DSP pipeline, the important sequence is:

ADC_STATUS



FIR_STATUS



FFT_STATUS



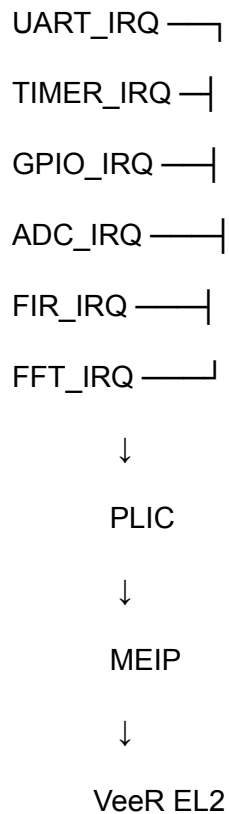
Result available

The processor can either **poll status registers** or use **interrupts** for events that require processor attention.

14. Interrupt-Based Control

Interrupts provide an alternative to continuous status polling.

The current interrupt path is:



A typical FFT completion sequence is:

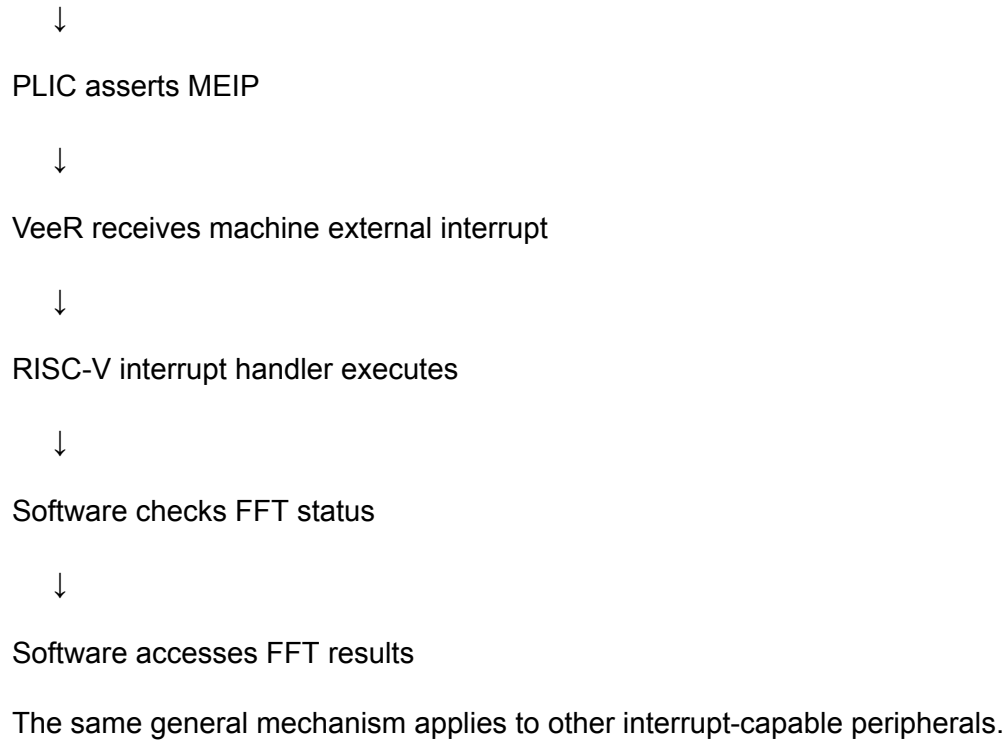
FFT completes



FFT_IRQ asserted



PLIC receives interrupt



15. DSP Result Control Flow

After FFT processing is complete, the processor needs access to the generated spectrum.

The intended sequence is:

FFT Accelerator

↓

Frequency-domain results

↓

Data Memory

↓

RISC-V LSU

↓

AHB-Lite



RISC-V software

The RISC-V can then process, interpret, or prepare the spectrum data for transmission.

The final communication path is:

FFT Results



Data Memory



RISC-V



UART



Host PC

This allows the Host PC to display or analyze the calculated frequency spectrum.

16. Control Path During Complete Operation

The complete operation can be divided into **configuration, acquisition, processing, completion, and result retrieval**.

Phase 1 — Configuration

RISC-V



UART configuration



Timer/GPIO configuration as required



ADC Interface configuration



FIR coefficient loading



FIR configuration



FFT configuration

Phase 2 — Enable

RISC-V



Enable ADC Interface



Enable FIR



Configure/enable FFT



Enable required interrupts

Phase 3 — Acquisition and Processing

The control path remains mostly supervisory while the data path operates:

External ADC



ADC Interface



FIR



FFT



Data Memory

Phase 4 — Completion

FFT completion



FFT_IRQ



PLIC



MEIP



RISC-V interrupt handler

Phase 5 — Result Retrieval

RISC-V



Read FFT results from Data Memory



Process/format results



UART



Host PC

17. Control Path and Data Path Relationship

The control and data paths operate together but have different purposes.

Control Path	Data Path
VeeR controls the system	Carries signal samples
Uses memory-mapped registers	Uses dedicated sample interfaces
AHB-Lite → APB	ADC → FIR → FFT
Configures FIR coefficients	Carries ADC samples
Configures FFT size/control	Carries filtered samples
Starts/stops processing	Carries FFT results
Reads status registers	Stores results in Data Memory

Handles interrupts/errors

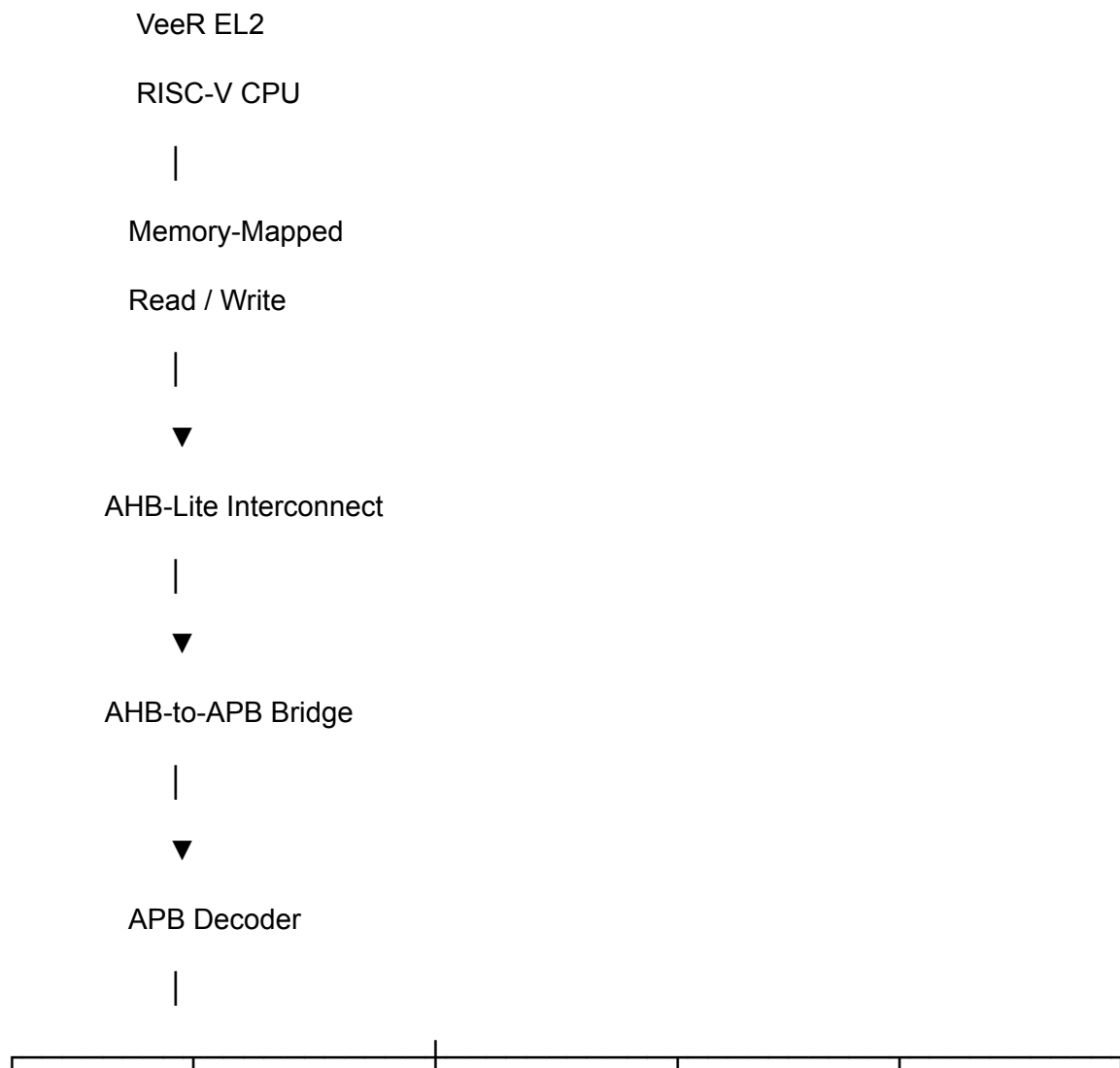
Performs DSP computation

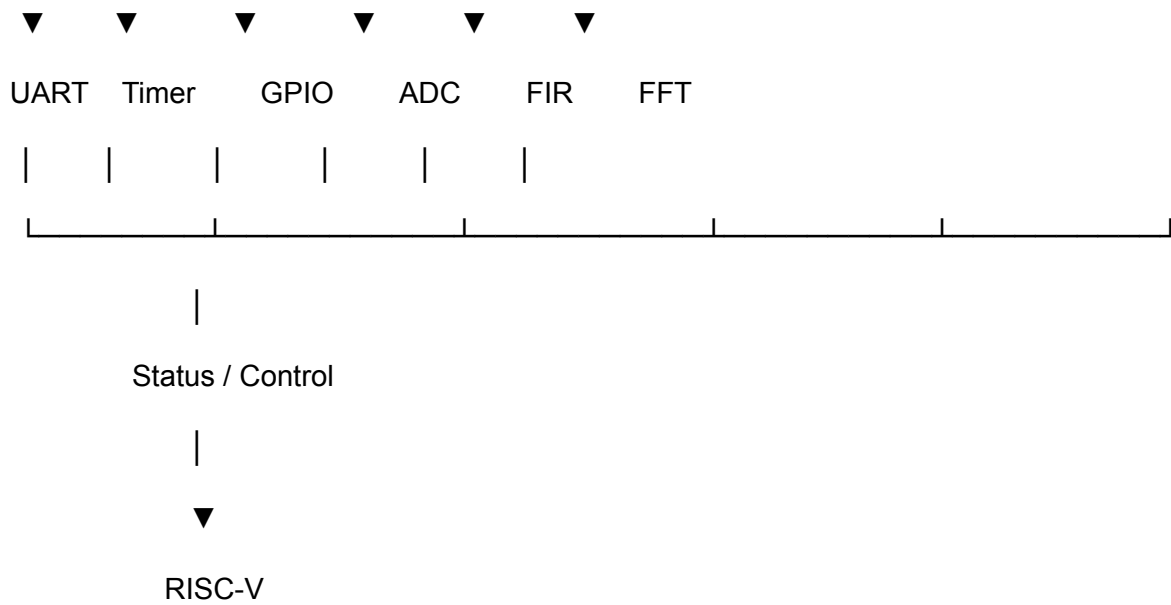
The fundamental principle is:

The control path determines how the hardware operates, while the data path performs and transports the actual signal processing.

18. Complete Control Path

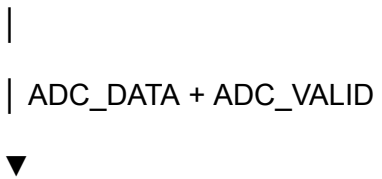
The complete control path of the proposed SoC can therefore be summarized as:



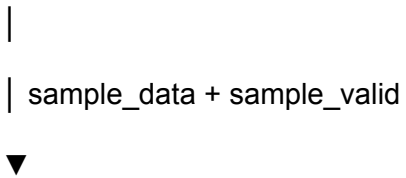


The corresponding DSP operation controlled by this path is:

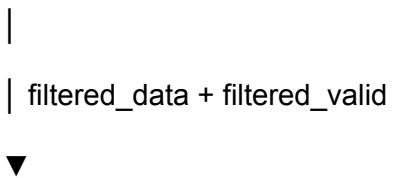
External ADC



ADC Interface



FIR Filter



FFT Accelerator



| FFT results



Data Memory



RISC-V



| UART



Host PC

Thus, **RISC-V acts as the system-level controller**, the **AHB-Lite/APB infrastructure provides the register-access mechanism**, and the **FIR/FFT hardware performs the computationally intensive DSP operations**. Interrupts provide event-driven notification to the processor, while status registers provide software visibility into the operating state of each IP.

Implementation note: The exact register bit fields, start/stop semantics, interrupt-clear mechanism, FFT output packing, and accelerator-to-memory interface must be finalized during the RTL microarchitecture phase. The above control path defines the intended architectural behavior and interfaces that the RTL implementation must realize.

DATA PATH

1. Purpose

The data path defines the **actual movement and processing of signal data** through the SoC, starting from the external analog signal and ending with the calculated frequency-spectrum results available to the RISC-V processor and Host PC.

The data path is separate from the control path. The control path configures and supervises the IPs, whereas the data path carries the actual signal samples and DSP results.

The primary data-processing path is:

External Analog Signal



External ADC



ADC Interface



FIR Filter



FFT Accelerator



Data Memory



RISC-V



UART



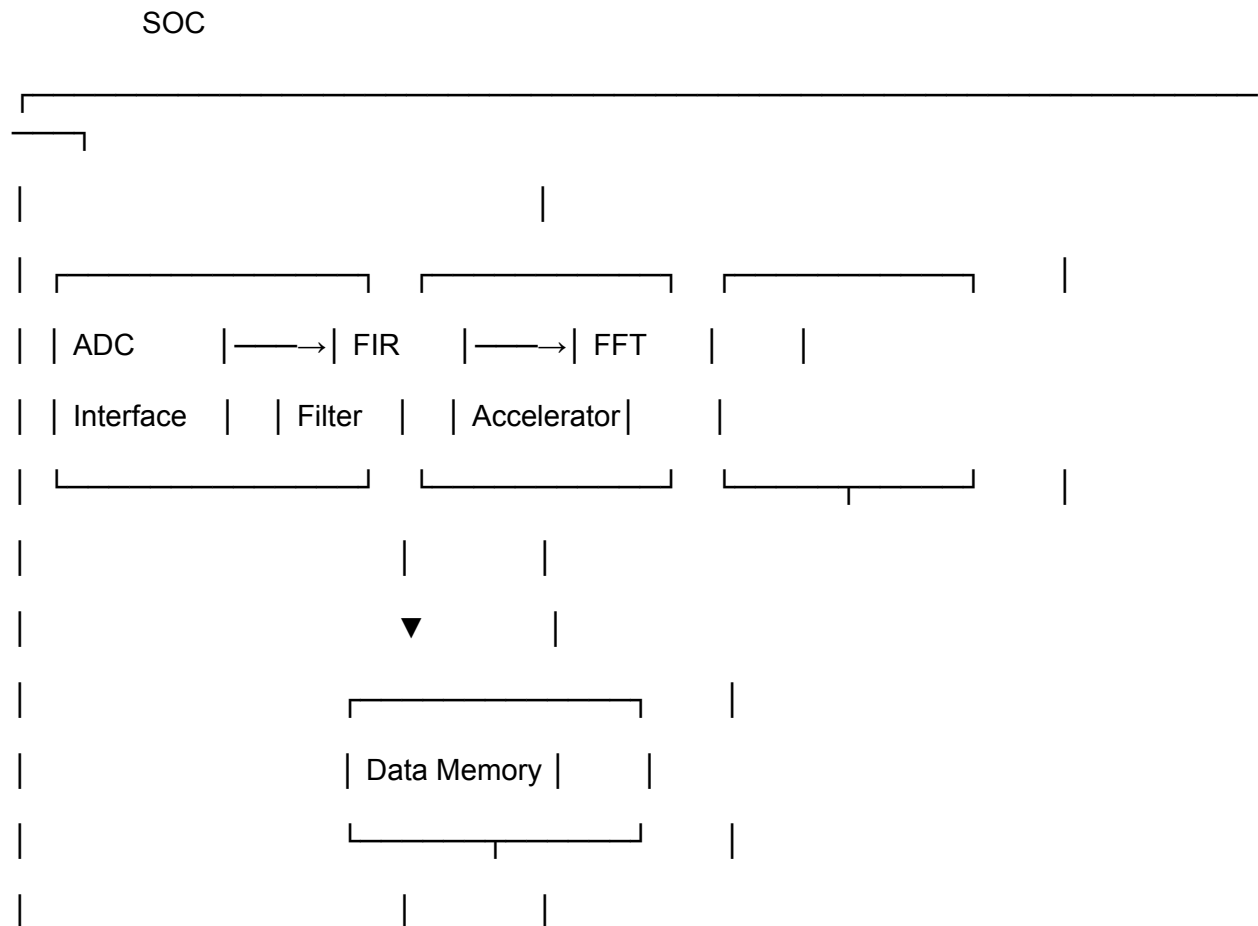
Host PC

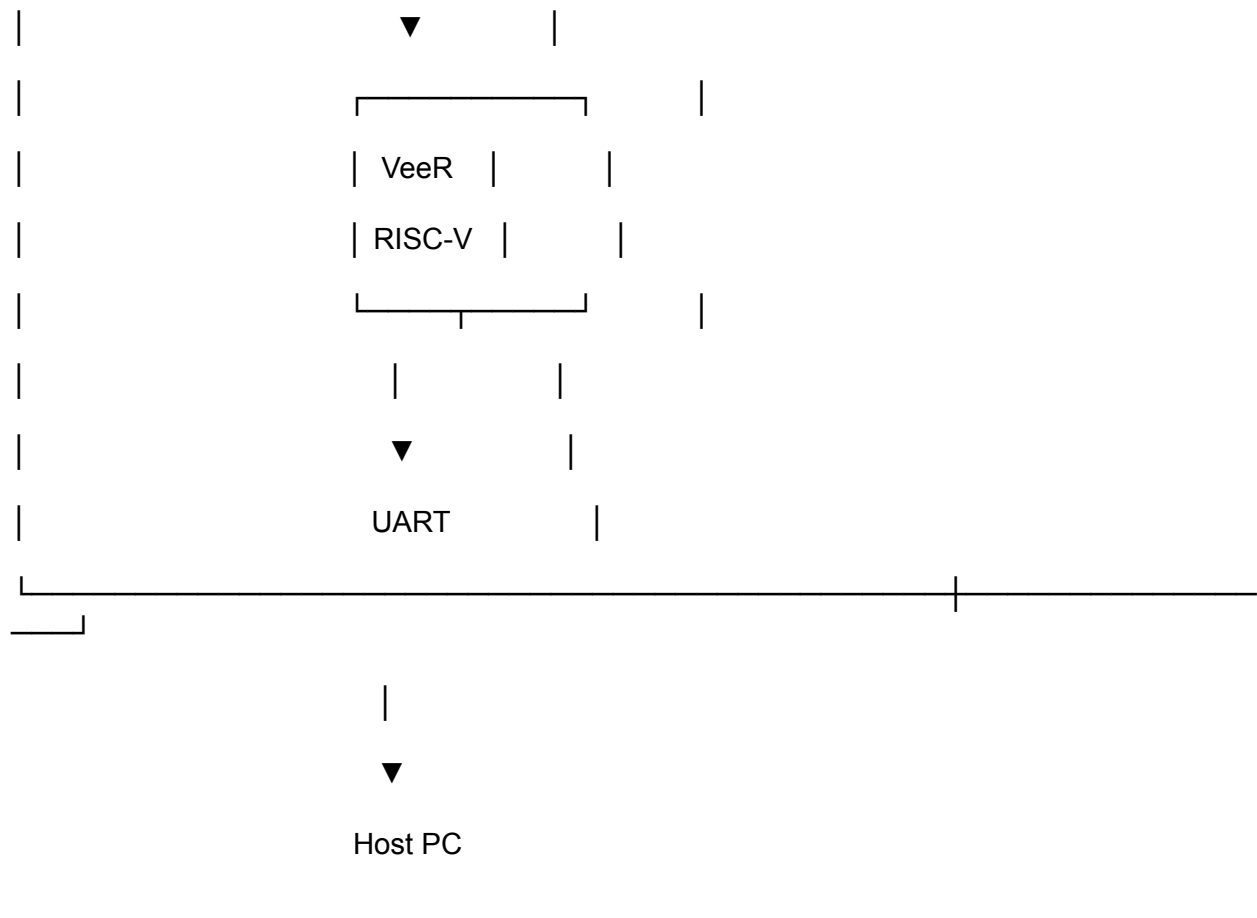
2. Overall Data Path Architecture

The complete data path consists of the following stages:

1. **External signal source**
2. **External ADC**
3. **ADC Interface**
4. **FIR Filter**
5. **FFT Accelerator**
6. **Data Memory**
7. **RISC-V processor**
8. **UART**
9. **Host PC**

The external ADC is outside the SoC boundary. All subsequent processing blocks are implemented inside the SoC.





3. External Signal and ADC

The data path begins with the signal that is to be analyzed.

The signal may originate from an external real-time application or signal source. The external ADC samples this analog signal and converts it into a digital representation.

The ADC is **not implemented inside the SoC**.

Analog Signal



External ADC



12-bit digital samples



SoC

The current interface specification uses:

ADC_DATA[11:0]

ADC_VALID

where:

- `ADC_DATA[11:0]` contains the 12-bit digitized sample.
- `ADC_VALID` indicates that the sample on `ADC_DATA` is valid.

The SoC therefore receives already-digitized data and does not perform analog-to-digital conversion itself.

4. ADC Interface

The ADC Interface is the first internal block in the DSP data path.

Its primary responsibility is to:

- Receive samples from the external ADC.
- Detect valid samples using `ADC_VALID`.
- Capture the 12-bit ADC sample.
- Present the sample to the FIR Filter using the internal DSP interface.

The data movement is:

External ADC

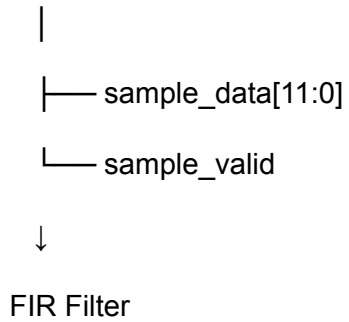


└─ ADC_DATA[11:0]

└─ ADC_VALID



ADC Interface



The ADC Interface therefore acts as a **boundary adapter between the external ADC and the internal DSP pipeline**.

It does not perform filtering or frequency analysis.

5. ADC Sample Representation

Each ADC sample is represented using the configured **12-bit sample width**:

ADC_DATA[11:0]

Therefore:

1 ADC sample = 12 bits

The ADC Interface passes the valid sample into the DSP pipeline when `sample_valid` indicates that the sample is available.

The sample stream can conceptually be represented as:

$x[0], x[1], x[2], x[3], \dots x[N-1]$

where each `x[n]` represents one digitized input sample.

6. ADC Interface → FIR Filter

The ADC Interface provides the digital sample stream to the FIR Filter.

The internal interface consists of:

sample_data[11:0]

sample_valid

The intended operation is:

ADC Interface

|
| sample_data[11:0]
| sample_valid



FIR Filter

When `sample_valid = 1`, the FIR accepts the corresponding sample.

When `sample_valid = 0`, the sample on `sample_data` is not considered a new valid input sample.

This valid-based transfer ensures that the FIR processes only valid ADC samples.

7. FIR Filter

The FIR Filter is the first custom DSP processing block.

Its purpose is to process the incoming time-domain ADC samples before they are supplied to the FFT Accelerator.

The FIR performs the filtering operation:

ADC samples



FIR Filter



Filtered samples

Conceptually, an FIR filter performs:

$$y[n] = \sum h[k] \times x[n-k]$$

where:

- $x[n]$ = input ADC sample
- $h[k]$ = FIR coefficient
- $y[n]$ = filtered output sample

The actual implementation will use the programmed FIR coefficients and the selected data widths defined during RTL design.

8. FIR Data Processing

The FIR receives the sample stream sequentially.

Conceptually:

$x[0]$ ┐

$x[1]$ ┐

$x[2]$ ┐ → FIR → $y[0], y[1], y[2], \dots$

$x[3]$ ┐

... ┘

The FIR maintains the required previous samples internally so that the required multiply-accumulate operations can be performed.

The FIR therefore converts the raw ADC sample stream into a **filtered time-domain sample stream**.

The filtered samples are then passed to the FFT Accelerator.

9. FIR → FFT Accelerator

The FIR output forms the input to the FFT Accelerator.

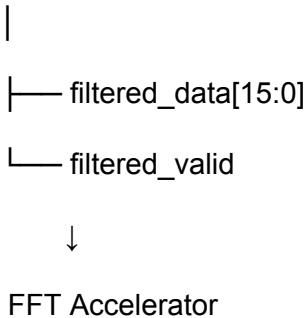
The current internal data interface is:

filtered_data[15:0]

filtered_valid

The intended data movement is:

FIR Filter



When `filtered_valid = 1`, the FFT accepts the corresponding filtered sample.

The **16-bit filtered-data width is an architectural interface choice** and must be maintained consistently between the FIR output and FFT input.

10. Purpose of FIR Before FFT

The FIR stage prepares the signal before frequency-domain analysis.

Depending on the selected application and final FIR design, the filter can be used to:

- Remove unwanted frequency components.
- Suppress high-frequency noise.
- Limit the signal bandwidth.
- Condition the signal before FFT processing.

Therefore, the processing chain is:

Raw sampled signal



FIR filtering



Conditioned signal



FFT



Frequency spectrum

The FIR is therefore a **signal-conditioning stage**, while the FFT is the **frequency-analysis stage**.

11. FFT Accelerator

The FFT Accelerator is the second custom DSP IP.

Its purpose is to convert a block of time-domain samples into their frequency-domain representation.

The FFT operates on a configured number of samples:

N time-domain samples



FFT



N frequency-domain results

The transform can conceptually be represented as:

$x[0], x[1], \dots, x[N-1]$



FFT

↓

$X[0], X[1], \dots, X[N-1]$

where:

- $x[n]$ represents the filtered time-domain samples.
 - $X[k]$ represents the corresponding frequency-domain FFT output.
-

12. FFT Data Collection

The FFT Accelerator receives filtered samples sequentially.

It must collect the required number of samples for one FFT frame before producing the corresponding frequency-domain output.

For an FFT size of N :

Sample 0

Sample 1

Sample 2

.

.

Sample $N-1$

↓

FFT processing

↓

Frequency-domain output

The FFT size is configured by software through the FFT control interface.

The exact supported FFT sizes must be finalized during RTL implementation.

13. FFT Output

The FFT produces frequency-domain results corresponding to different frequency bins.

Conceptually:

Time Domain

$x[0] \dots x[N-1]$

↓

FFT

↓

Frequency Domain

$X[0] \dots X[N-1]$

Each FFT bin represents a particular frequency component of the input signal.

The frequency represented by a bin is determined by the sampling frequency and FFT size:

$$\Delta f = F_s / N$$

where:

- F_s = ADC sampling frequency
- N = FFT size
- Δf = frequency resolution

The FFT output therefore forms the basis of the spectrum analyzer's frequency-domain result.

14. FFT → Data Memory

After FFT processing, the generated frequency-domain results are stored in the SoC's Data Memory.

The conceptual path is:

FFT Accelerator

↓
FFT output data

↓
Data Memory

Data Memory provides storage for the generated FFT results so that the RISC-V processor can subsequently access them.

The memory therefore acts as the **storage point between hardware DSP processing and software-level result handling**.

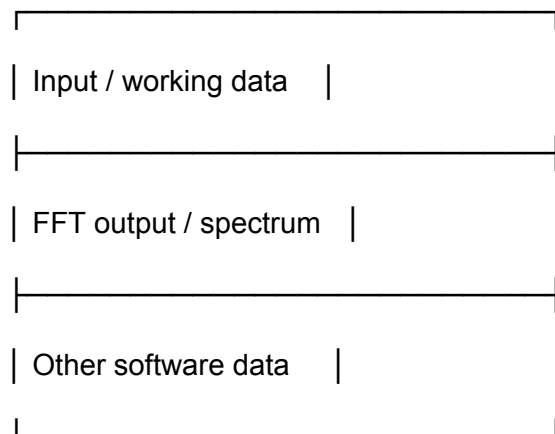
15. Data Memory Organization

The Data Memory is shared system memory accessible through the AHB-Lite system infrastructure.

The FFT output is stored in a designated memory region.

Conceptually:

Data Memory



The exact memory partitioning and FFT output base address must be finalized during implementation.

The specification should therefore define the required memory capacity and reserved FFT result region before RTL integration.

16. Data Memory → RISC-V

Once the FFT results have been stored, the RISC-V accesses the results using normal memory read operations.

The path is:

Data Memory



AHB-Lite



VeeR EL2 RISC-V

The processor can then:

- Read FFT results.
- Calculate magnitude/power if those operations are assigned to software.
- Determine dominant frequency components.
- Format the spectrum results.
- Prepare data for transmission to the Host PC.

The exact division between hardware FFT output processing and software post-processing must be finalized as part of the implementation.

17. RISC-V → UART → Host PC

After obtaining the required spectrum information, the RISC-V sends the results through UART.

RISC-V



UART



UART_TX



Host PC

The Host PC can then receive the spectrum data and display it in a suitable frequency-spectrum format.

The Host PC is therefore primarily the **external visualization/monitoring interface**, rather than part of the SoC's DSP processing path.

18. Complete DSP Data Path

The complete hardware DSP data path is:

External Analog Signal



External ADC



ADC_DATA[11:0]

ADC_VALID



ADC Interface



sample_data[11:0]

sample_valid



FIR Filter



filtered_data[15:0]

filtered_valid



FFT Accelerator



FFT Results

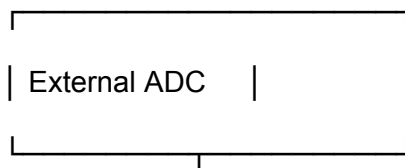
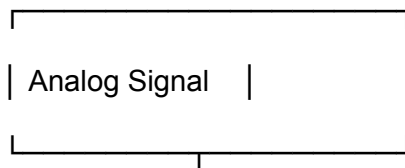


Data Memory

This is the **primary data path of the spectrum analyzer**.

19. Complete End-to-End Data Flow

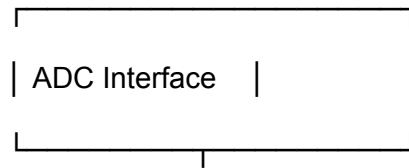
Including software handling and external communication:



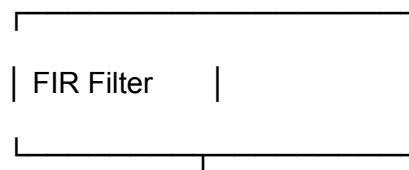
| ADC_DATA[11:0]

| ADC_VALID

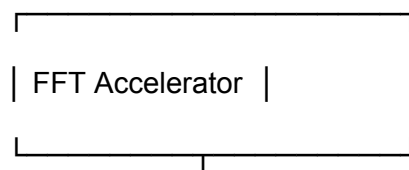




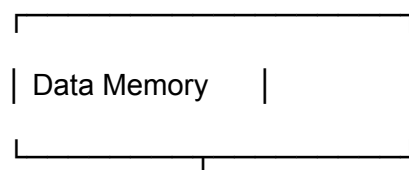
| sample_data[11:0]
| sample_valid

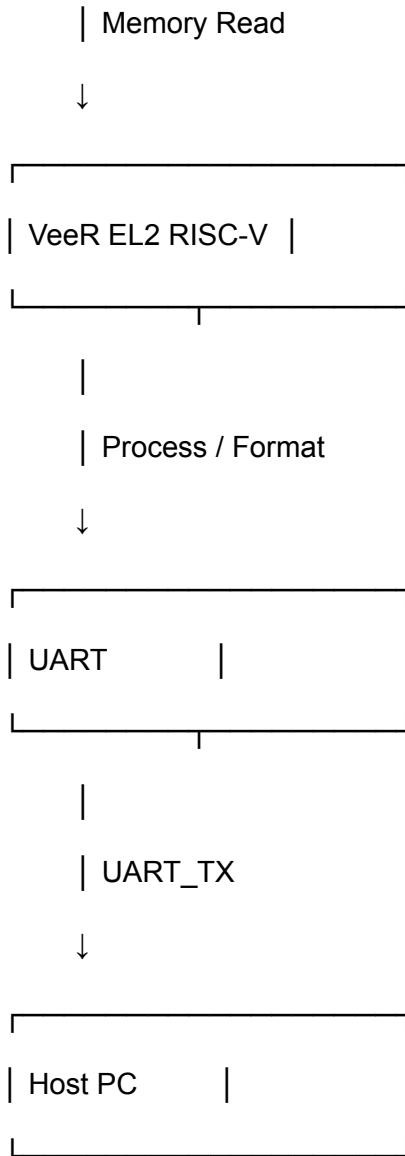


| filtered_data[15:0]
| filtered_valid



| FFT Results





20. Data Width Progression

The currently defined data widths through the primary DSP path are:

Stage	Data	Width
-------	------	-------

External ADC → ADC Interface	<code>ADC_DATA[11:0]</code>	12 bits
ADC Interface → FIR	<code>sample_data[11:0]</code>	12 bits
FIR → FFT	<code>filtered_data[15:0]</code>	16 bits
FFT → Data Memory	FFT result	To be finalized based on FFT representation
Data Memory → RISC-V	Memory word	System memory interface width

The FIR output is wider than the ADC input to accommodate the increased numerical range resulting from filtering and multiply-accumulate operations.

The final internal arithmetic widths, FFT input/output widths, complex-data representation, and memory packing must be explicitly fixed during RTL microarchitecture.

21. Streaming vs Stored Data

The first part of the DSP path is fundamentally a **streaming sample path**:

ADC Interface

↓

FIR

↓

FFT

Samples are processed as they arrive rather than requiring the RISC-V to individually move every sample between IPs.

The FFT operates on a block/frame of samples and produces the corresponding frequency-domain results.

The output is then stored in Data Memory:

Streaming samples



FIR



FFT frame



FFT results



Data Memory



Software access

This architecture keeps the computationally intensive sample processing in hardware and avoids unnecessary processor involvement in every sample transfer.

22. Role of RISC-V in the Data Path

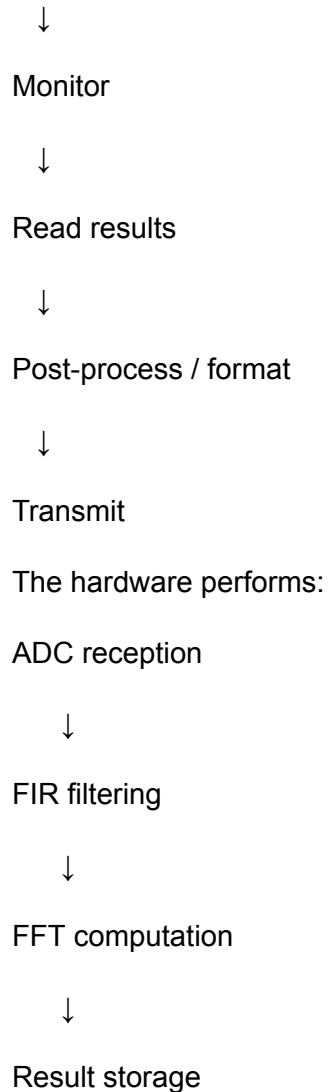
The RISC-V is **not the primary sample-processing engine**.

Its role is primarily:

Configure



Start



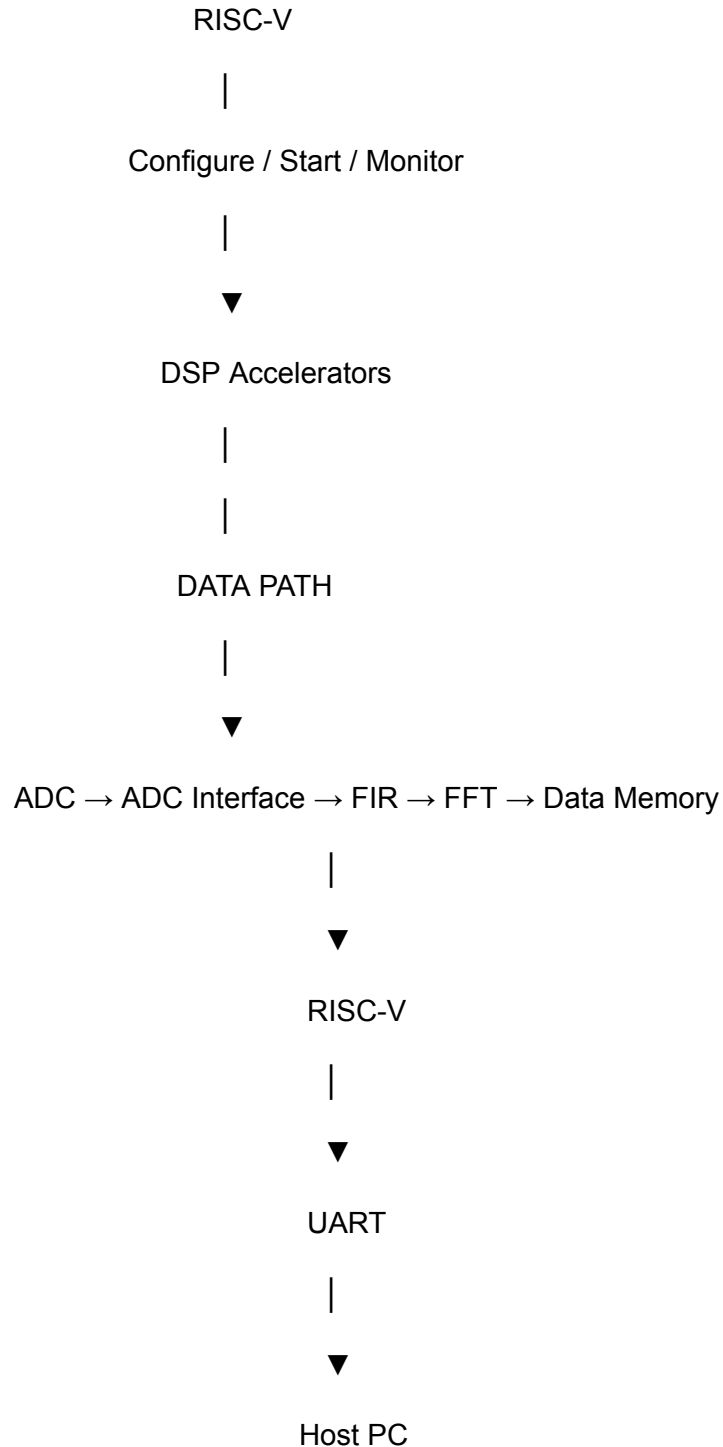
This separation allows the processor to perform system-level control while the custom DSP accelerators perform the computationally intensive operations.

23. Data Path and Control Path Interaction

The two paths operate together as follows:

CONTROL PATH





The processor configures the DSP blocks through the control path before data processing begins. The actual samples then move through the hardware data path. Once processing completes, the processor retrieves the results and handles the external communication.

24. Data Path Operational Sequence

The complete operational sequence is:

Step 1 — Signal Acquisition

Analog input



External ADC



12-bit digital samples

Step 2 — Sample Reception

ADC



ADC Interface



sample_data + sample_valid

Step 3 — Filtering

ADC samples



FIR Filter



Filtered samples

Step 4 — FFT Processing

Filtered sample frame



FFT Accelerator



Frequency-domain bins

Step 5 — Result Storage

FFT results



Data Memory

Step 6 — Software Retrieval

Data Memory



RISC-V

Step 7 — External Output

RISC-V



UART



Host PC

25. Data Path Design Requirements

The RTL implementation shall maintain the following architectural requirements:

1. The external ADC shall provide digitized samples to the SoC through `ADC_DATA[11:0]` and `ADC_VALID`.
2. The ADC Interface shall capture valid ADC samples and provide them to the FIR Filter.
3. The FIR Filter shall process the incoming samples using the programmed coefficients.
4. The FIR shall provide filtered samples to the FFT Accelerator.

5. The FFT Accelerator shall collect the configured number of samples and perform the FFT.
 6. FFT results shall be made available to the Data Memory.
 7. The RISC-V shall be able to access the stored FFT results.
 8. The RISC-V shall be able to transmit the required results to the Host PC through UART.
 9. Sample-level DSP processing shall not depend on software moving every individual sample.
 10. Data widths and numerical representations shall remain consistent between connected DSP blocks.
 11. The exact FIR arithmetic width, FFT internal precision, FFT output format, and memory packing shall be finalized before RTL implementation.
 12. The sampling frequency F_s and FFT size N shall be fixed as part of the final application specification because they determine the frequency resolution $\Delta f = F_s/N$.
-

26. Final Data Path Summary

The fundamental data-flow architecture of the proposed spectrum-analyzer SoC is:

EXTERNAL WORLD

|



Analog Signal

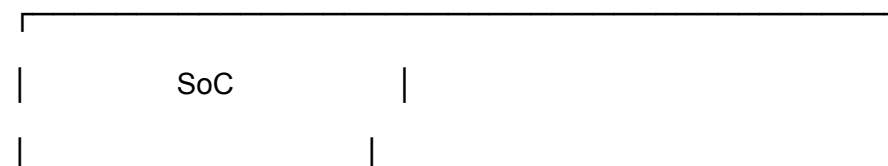
|

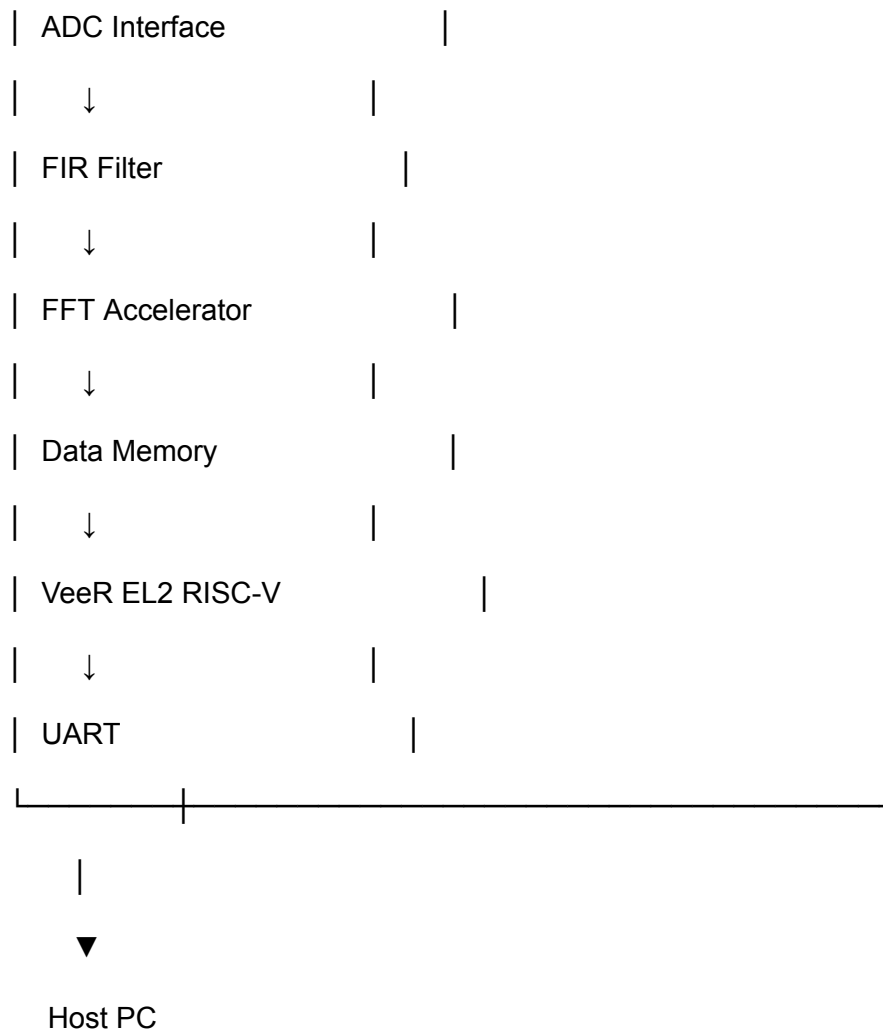


External ADC

|

| 12-bit digital samples





In one sentence:

The data path receives digitized samples from the external ADC, streams them through the ADC Interface and FIR Filter, processes a sample block using the FFT Accelerator to obtain its frequency-domain representation, stores the FFT results in Data Memory, and allows the RISC-V processor to retrieve, process, and transmit the required spectrum information to the Host PC through UART.

REQUIRED CHANGES

1. FFT-to-Data-Memory Interface Must Be Explicitly Defined

Location in Document

Signal List → Block 17C: FFT Accelerator ↔ Data Memory

- **Required Change:** Retain the dedicated FFT-to-Data-Memory interface (`fft_mem_addr`, `fft_mem_wdata`, `fft_mem_we`, `fft_mem_valid`, `fft_mem_ready`) and explicitly define its transaction behavior.
 - **Reason:** FFT results must be stored in Data Memory before they can be accessed by the RISC-V processor through AHB-Lite.
 - **Final Requirement:** The specification shall clearly define the path **FFT → Data Memory → RISC-V** and the corresponding memory-write handshake.
-

2. Data Memory Must Support Both FFT and RISC-V Access

Location in Document

Data Path → Data Memory Organization / FFT → Data Memory

- **Required Change:** Define how Data Memory handles access from both the FFT accelerator and RISC-V/AHB-Lite.
 - **Reason:** FFT writes spectrum results to Data Memory, while RISC-V subsequently reads those results; therefore, memory access conflicts must be handled.
 - **Final Requirement:** Specify whether Data Memory uses **dual-port memory, arbitration, or another access mechanism**, including priority and simultaneous-access behavior.
-

3. AHB-Lite 64-bit ↔ APB 32-bit Data-Width Conversion Must Be Frozen

Location in Document

Signal List → Block 15: AHB-Lite ↔ AHB-to-APB Bridge

- **Required Change:** Define how 64-bit AHB-Lite transactions are converted to 32-bit APB transactions, including data selection, byte enables, and read-data placement.
 - **Reason:** The current specification defines AHB as 64-bit and APB as 32-bit, but the exact conversion behavior is not yet fixed.
 - **Final Requirement:** Freeze the conversion rules, including `HSIZE`, byte enables, `HWDATA[63:0]` handling, `PRDATA[31:0]` placement, and unsupported transfer behavior.
-

4. DSP Numerical Specifications Must Be Frozen Before RTL

Location in Document

Signal List → FIR Filter / FFT Accelerator

- **Required Change:** Finalize FIR taps, coefficient width, accumulator/output widths, fixed-point format, and FFT input/output precision and scaling.
 - **Reason:** These parameters determine DSP accuracy, overflow behavior, hardware resources, interface widths, and final spectrum correctness.
 - **Final Requirement:** All FIR and FFT numerical formats shall be frozen before RTL implementation.
-

5. FFT Output-to-Memory Packing Must Be Explicitly Defined

Location in Document

Signal List → Block 17C: FFT Accelerator ↔ Data Memory

- **Required Change:** Define how the complex FFT output (Real + Imaginary) is stored using the currently defined `fft_mem_wdata[31:0]`.
- **Reason:** A complex FFT result may require more than one 32-bit memory word, so the current interface does not uniquely define the memory layout.
- **Final Requirement:** Freeze the **real/imaginary widths, memory word width, packing format, address increment, number of words per FFT bin, and result-buffer layout**.